LinkCode — 深度实现细节调研报告

LinkCode VS Code 插件 — 深度实现细节调研报告

> **版本**: v1.0 | **日期**: 2026-04-21 | **状态**: 技术调研完成

目录

1. [VS Code Extension API 深度调研](#1-vs-code-extension-api-深度调研)
2. [竞品技术实现深度分析](#2-竞品技术实现深度分析)
3. [多模型路由引擎详细设计](#3-多模型路由引擎详细设计)
4. [代码补全的完整技术方案](#4-代码补全的完整技术方案)
5. [Chat 面板完整技术方案](#5-chat-面板完整技术方案)
6. [项目索引和 RAG 方案](#6-项目索引和-rag-方案)
7. [Smoothlink API 集成细节](#7-smoothlink-api-集成细节)
8. [安全和隐私方案](#8-安全和隐私方案)
9. [性能优化方案](#9-性能优化方案)
10. [发布和分发方案](#10-发布和分发方案)

---

1. VS Code Extension API 深度调研

1.1 Inline Completion API (`vscode.InlineCompletionItemProvider`)

API 版本要求: VS Code 1.68+ (Stable API since 1.68)

核心接口:

代码块

```
import * as vscode from 'vscode';

class LinkCodeCompletionProvider implements vscode.InlineCompletionItemProvider {

  async provideInlineCompletionItems(
    document: vscode.TextDocument,
    position: vscode.Position,
    context: vscode.InlineCompletionContext,
    token: vscode.CancellationToken
  ): Promise<vscode.InlineCompletionItem[] | vscode.InlineCompletionList | undefined> {

    // context.triggerKind:
    // - InlineCompletionTriggerKind.Invoke (0): 用户主动触发 (Ctrl+Space)
    // - InlineCompletionTriggerKind.Automatic (1): 自动触发 (打字时)

    // context.selectedCompletionInfo: 如果 IntelliSense 菜单打开，包含选中项信息

    if (token.isCancellationRequested) return undefined;

    const prefix = document.getText(new vscode.Range(new vscode.Position(0, 0), position));
    const suffix = document.getText(new vscode.Range(position, document.positionAt(document.getText().length)));

    const completion = await this.fetchCompletion(prefix, suffix, token);

    if (!completion || token.isCancellationRequested) return undefined;

    return [
      new vscode.InlineCompletionItem(
        completion,
        new vscode.Range(position, position) // insertText 的范围
      )
    ];
  }
}

```

```
// 注册
const disposable = vscode.languages.registerInlineCompletionItemProvider(
  { pattern: '**' }, // 所有文件
  new LinkCodeCompletionProvider()
);
```

关键限制与注意事项:

限制	详情	影响
单个 Provider	VS Code 同时只显示一个 inline completion provider 的结果	与 Copilot 冲突时需要用户禁用其中一个
无优先级控制	不能设置 provider 优先级	如果用户同时安装了 Copilot，行为不可预测
CancellationToken	VS Code 会在用户继续输入时 cancel 之前的请求	必须正确处理 token.isCancellationRequested
无多行高亮控制	Ghost Text 样式由 VS Code 控制，无法自定义颜色	不能像 Cursor 那样做差异化 UI
最大返回数	可返回数组，但 VS Code 只展示第一个	多候选需要自己在服务端选最优
Tab 键冲突	Tab 接受补全 vs Tab 缩进存在冲突	需要正确配置 `editor.tabCompletion`

性能注意事项:

- `provideInlineCompletionItems` 每次按键都会被调用（Automatic 模式）
- 必须在 Extension 层做 debounce（推荐 150-300ms）
- API 本身不提供 debounce 机制
- 返回 `undefined` 时 VS Code 不会显示任何 ghost text
- `CancellationToken` 必须传递到下游 API 调用

⚠ Pitfall:

- 如果 `provideInlineCompletionItems` 抛出异常，VS Code 不会有明显错误提示，补全会静默失败
- 在 `selectedCompletionInfo` 存在时（IntelliSense 打开），需要决定是否仍提供补全

✅ 推荐方案: 直接使用 `InlineCompletionItemProvider`，不需要 LSP。对于代码补全场景足够。

1.2 Webview API 限制

核心 API:

代码块

```
// 创建 Webview Panel (独立面板)
const panel = vscode.window.createWebviewPanel(
  'linkcode.chat',
  'LinkCode Chat',
  vscode.ViewColumn.Beside,
  {
    enableScripts: true,
    retainContextWhenHidden: true, // 重要: 保持状态
    localResourceRoots: [
      vscode.Uri.joinPath(context.extensionUri, 'webview-dist')
    ]
  }
);

// 或注册为侧边栏 View (推荐用于 Chat)
class ChatViewProvider implements vscode.WebviewViewProvider {
  resolveWebviewView(
    webviewView: vscode.WebviewView,
    context: vscode.WebviewViewResolveContext,
    token: vscode.CancellationToken
  ) {
    webviewView.webview.options = {
      enableScripts: true,
      localResourceRoots: [this.extensionUri]
    };
    webviewView.webview.html = this.getHtmlForWebview(webviewView.webview);
  }
}
```

CSP (Content Security Policy) 限制:

代码块

```
function getHtmlForWebview(webview: vscode.Webview): string {
  const nonce = getNonce(); // 随机 nonce
  const styleUri = webview.asWebviewUri(vscode.Uri.joinPath(extensionUri, 'dist', 'style.css'));
  const scriptUri = webview.asWebviewUri(vscode.Uri.joinPath(extensionUri, 'dist', 'webview.js'));

  return `<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta http-equiv="Content-Security-Policy" content="
    default-src 'none';
    style-src ${webview.cspSource} 'unsafe-inline';
    script-src 'nonce-${nonce}';
    font-src ${webview.cspSource};
    img-src ${webview.cspSource} https: data:;
    connect-src https://api.smoothlink.ai;
  ">
  <link href="${styleUri}" rel="stylesheet">
</head>
<body>
  <div id="root"></div>
  <script nonce="${nonce}" src="${scriptUri}"></script>
</body>
</html>`;
}
```

CSP 关键限制:

- ❌ 不能使用 `eval()` 或 `new Function()` — 影响某些 JS 库
- ❌ 不能加载外部 CDN 脚本 — 所有 JS 必须 bundle
- ❌ 内联 `<script>` 必须有 nonce — React/Vue 的 inline style 需要 `'unsafe-inline'`
- ✅ `connect-src` 可以允许直接从 Webview 发 API 请求（但建议通过 Extension Host 代理）

消息传递机制:

代码块

```
// Extension Host → Webview
panel.webview.postMessage({ type: 'streamChunk', data: { content: '...', done: false } });

// Webview → Extension Host
// Webview 端:
const vscode = acquireVsCodeApi();
vscode.postMessage({ type: 'sendMessage', data: { text: 'hello', files: [] } });

// Extension Host 端:
panel.webview.onDidReceiveMessage(message => {
  switch (message.type) {
    case 'sendMessage':
      this.handleChatMessage(message.data);
      break;
  }
});
```

状态持久化:

代码块

```
// Webview 端使用 VS Code State API
const vscode = acquireVsCodeApi();

// 设置状态（在 Webview 被 hidden 时保留）
vscode.setState({ chatHistory: [...], scrollPosition: 100 });

// 恢复状态
const previousState = vscode.getState();

// 注意: retainContextWhenHidden = true 时，整个 DOM 都保留
// 但会占用更多内存
```

⚠ Pitfall:

- `WebviewView`（侧边栏）没有 `retainContextWhenHidden` 选项！切换到其他 panel 时 Webview 会被销毁

- 解决方案：使用 `vscode.setState()` 保存状态，并在 `resolveWebviewView` 中恢复

- Webview 与 Extension Host 通信是异步的，大量消息时可能有延迟

- Webview 无法直接访问 Node.js API 或文件系统

✅ 推荐: 使用 `WebviewViewProvider` 注册到侧边栏 Activity Bar，Chat 面板用 React 构建，通过 `postMessage` 与 Extension Host 通信。

1.3 VS Code Secret Storage API

代码块

```
// 存储 API Key
export class AuthManager {
  private static readonly KEY_SMOOTHLINK_API = 'linkcode.smoothlink.apiKey';
  private static readonly KEY_SMOOTHLINK_TOKEN = 'linkcode.smoothlink.accessToken';

  constructor(private secrets: vscode.SecretStorage) {}

  async storeApiKey(apiKey: string): Promise<void> {
    await this.secrets.store(AuthManager.KEY_SMOOTHLINK_API, apiKey);
  }

  async getApiKey(): Promise<string | undefined> {
    return await this.secrets.get(AuthManager.KEY_SMOOTHLINK_API);
  }

  async deleteApiKey(): Promise<void> {
    await this.secrets.delete(AuthManager.KEY_SMOOTHLINK_API);
  }

  // 监听变化
  onDidChange(callback: (e: vscode.SecretStorageChangeEvent) => void): vscode.Disposable {
    return this.secrets.onDidChange(callback);
  }
```

```
}

// 在 activate 中使用
export function activate(context: vscode.ExtensionContext) {
  const authManager = new AuthManager(context.secrets);
}
```

Secret Storage 技术细节:

平台	底层实现	安全性
macOS	Keychain	系统级加密，需要用户授权
Windows	Windows Credential Manager (DPAPI)	用户级加密
Linux	libsecret (GNOME Keyring / KWallet)	依赖桌面环境
VS Code Web	浏览器 LocalStorage (降级)	⚠ 安全性较低

限制:
- 存储大小：无明确限制，但建议 < 1KB per secret（存 API Key 足够）
- 仅支持 string 类型，复杂数据需要 JSON.stringify
- 无法在 Webview 中直接访问，必须通过 Extension Host 中转
- 在无桌面环境的 Linux 上可能失败（Headless Server）

⚠ Pitfall:
- Linux 无 GNOME/KDE 时，Secret Storage 可能回退到明文存储
- 多个 VS Code 窗口共享同一个 Secret Storage

1.4 Language Server Protocol (LSP)

结论：MVP 阶段不需要 LSP

| 场景 | 是否需要 LSP | 原因 |

|------|----------|------|

| AI 代码补全 | ❌ | InlineCompletionItemProvider 足够 |

| Chat 面板 | ❌ | Webview + postMessage |

| 代码导航/跳转 | ❌ | 使用 VS Code 内置 |

| @symbol 引用 | ❌ | 使用 `vscode.commands.executeCommand('vscode.executeDocumentSymbolProvider')` |

| 大规模重构 | ⚠ 可能 | V2 Agent 模式可能需要 |

什么时候用 LSP:

- 需要支持多 IDE (JetBrains) 时，用 LSP 可以复用语言分析逻辑

- 需要做复杂的代码分析（如 Project-wide 类型推断）时

- P2 阶段的 JetBrains 支持可以考虑引入 LSP

1.5 Extension Host 进程模型与内存限制

进程架构:

代码块

```
VS Code Main Process (Electron)
├── Renderer Process (UI)
├── Extension Host Process (Node.js) ← 我们的代码跑在这里
│   ├── LinkCode Extension
│   ├── Other Extensions...
│   └── 共享同一个 Node.js 进程
├── Extension Host Process (Web Worker) ← Web extensions
└── File Watcher Process
```

内存限制:

- Extension Host 是一个独立的 Node.js 进程

- 默认 V8 堆内存限制：~1.4GB (64-bit)

- 所有 Extension 共享这个限制

- VS Code 会在 Extension Host 占用过多内存时显示警告

- **建议 LinkCode 控制在 < 200MB**

性能监控:

代码块

```
// 可以用 process.memoryUsage() 监控
setInterval(() => {
  const usage = process.memoryUsage();
  console.log(`RSS: ${(usage.rss / 1024 / 1024).toFixed(1)}MB`);
  console.log(`Heap: ${(usage.heapUsed / 1024 / 1024).toFixed(1)}MB`);
}, 60000);
```

⚠ Pitfall:

- Extension Host 崩溃会影响所有 extension

- 长时间阻塞 Extension Host 主线程会导致整个 VS Code UI 卡顿

- 大量计算（如 RAG 索引）应该放到 Worker Thread 或子进程

1.6 代码上下文获取

方案对比:

| 方案 | 优点 | 缺点 | 推荐 |

|------|------|------|------|

| VS Code 内置 API | 零依赖，直接可用 | 粒度有限 | ✅ MVP |

| Tree-sitter (web-tree-sitter) | 精确 AST，跨语言 | 需要额外 WASM 文件 (~2MB/语言) | ✅ P1 |

| VS Code Semantic Tokens API | 利用已有语言服务 | 依赖语言扩展 | ⚠ 辅助 |

方案 A：VS Code 内置 API（MVP 推荐）

代码块

```
// 1. 获取文档符号
const symbols = await vscode.commands.executeCommand<vscode.DocumentSymbol[]>(
  'vscode.executeDocumentSymbolProvider',
  document.uri
);

// 2. 找到光标所在的函数/类
function findContainingSymbol(
  symbols: vscode.DocumentSymbol[],
  position: vscode.Position
): vscode.DocumentSymbol | undefined {
  for (const symbol of symbols) {
    if (symbol.range.contains(position)) {
      // 递归查找最内层
      const child = findContainingSymbol(symbol.children, position);
      return child || symbol;
    }
  }
  return undefined;
}

// 3. 获取 import 语句
function getImports(document: vscode.TextDocument): string[] {
  const text = document.getText();
  const importRegex = /^(import\s+.*|const\s+.*=\s*require\(.*\)|from\s+.*import\s+.*)/gm;
  const matches = [];
  let match;
  while ((match = importRegex.exec(text)) !== null) {
    matches.push(match[0]);
  }
  return matches;
}

// 4. 获取当前打开的相关文件
function getOpenEditorContents(): Map<string, string> {
  const contents = new Map<string, string>();
  for (const tabGroup of vscode.window.tabGroups.all) {
    for (const tab of tabGroup.tabs) {
      if (tab.input instanceof vscode.TabInputText) {
        const uri = tab.input.uri;
        // 获取已打开文件的内容
```

```
        const doc = vscode.workspace.textDocuments.find(d => d.uri.toString() === uri.toString());
        if (doc) {
          contents.set(uri.fsPath, doc.getText());
        }
      }
    }
  }
  return contents;
}
```

方案 B：Tree-sitter（P1 推荐）

代码块

```
import Parser from 'web-tree-sitter';

class TreeSitterContext {
  private parser: Parser | null = null;
  private trees: Map<string, Parser.Tree> = new Map();

  async init() {
    await Parser.init();
    this.parser = new Parser();

    // 加载语言 (WASM files bundled with extension)
    const TypeScript = await Parser.Language.load(
      path.join(__dirname, 'tree-sitter-typescript.wasm')
    );
    this.parser.setLanguage(TypeScript);
  }

  parseDocument(uri: string, content: string): Parser.Tree {
    const tree = this.parser!.parse(content);
    this.trees.set(uri, tree);
    return tree;
  }

  // 获取光标位置的函数体
  getFunctionAtPosition(uri: string, line: number, column: number): string | null {
    const tree = this.trees.get(uri);
    if (!tree) return null;
```

```

    let node = tree.rootNode.descendantForPosition({ row: line, column });
    while (node) {
      if (['function_declaration', 'method_definition', 'arrow_function'].includes(node.type)) {
        return node.text;
      }
      node = node.parent!;
    }
    return null;
  }

  // 增量更新（性能关键）
  updateDocument(uri: string, content: string, edit: Parser.Edit): Parser.Tree {
    const oldTree = this.trees.get(uri);
    if (oldTree) {
      oldTree.edit(edit);
      const newTree = this.parser!.parse(content, oldTree);
      this.trees.set(uri, newTree);
      return newTree;
    }
    return this.parseDocument(uri, content);
  }
}
```

Tree-sitter WASM 文件大小参考:

- TypeScript: ~1.8MB
- JavaScript: ~300KB
- Python: ~400KB
- Java: ~600KB
- Go: ~500KB
- 总计（10种主流语言）: ~8-10MB

✅ 推荐路径:

- MVP 阶段用 VS Code 内置 API + 简单正则
- P1 阶段引入 Tree-sitter 获取精确 AST 上下文

---

2. 竞品技术实现深度分析

2.1 Continue.dev 架构分析

仓库: `github.com/continuedev/continue` (Apache 2.0)

技术栈: TypeScript + React

架构概览:

代码块

```
continue/
├── core/              # 核心逻辑（模型无关）
│   ├── autocomplete/  # 代码补全引擎
│   │   ├── CompletionProvider.ts    # 主入口
│   │   ├── context/                  # 上下文构建
│   │   │   ├── ImportDefinitionsService.ts
│   │   │   ├── RootPathContext.ts
│   │   │   └── RecentlyEditedTracker.ts
│   │   ├── filtering.ts             # 补全过滤/后处理
│   │   ├── cache.ts                 # LRU 缓存
│   │   └── charStream.ts            # 流式处理
│   ├── context/       # 上下文 Provider 系统
│   │   ├── providers/
│   │   │   ├── FileContextProvider.ts    # @file
│   │   │   ├── CodebaseContextProvider.ts # @codebase (RAG)
│   │   │   ├── FolderContextProvider.ts   # @folder
│   │   │   └── URLContextProvider.ts      # @url
│   │   └── retrieval/
│   │       ├── LanceDbIndex.ts      # 向量存储
│   │       └── ChunkCodebaseIndex.ts # 代码分块索引
│   ├── llm/           # LLM 抽象层
│   │   ├── BaseLLM.ts
│   │   ├── llms/
│   │   │   ├── OpenAI.ts
│   │   │   ├── Anthropic.ts
│   │   │   └── Ollama.ts
│   │   └── stream.ts
│   └── indexing/      # 代码索引系统
```

```
│       ├── CodebaseIndexer.ts
│       ├── chunk.ts
│       └── embeddings.ts
├── extensions/
│   └── vscode/        # VS Code extension
│       ├── src/
│       │   ├── extension.ts
│       │   ├── autocomplete/
│       │   │   └── InlineCompletionProvider.ts
│       │   ├── webviews/             # React UI
│       │   └── ContinueGUIWebviewProvider.ts
│       └── package.json
└── gui/               # React 前端 (Chat UI)
    ├── src/
    │   ├── components/
    │   │   ├── Chat.tsx
    │   │   ├── StepContainer.tsx     # 消息渲染
    │   │   └── CodeBlock.tsx         # 代码块
    │   └── hooks/
    └── package.json
```

Continue.dev 补全关键实现:

1. **上下文构建**: 使用滑动窗口 + 相关文件片段
   - 光标前 ~2000 tokens
   - 光标后 ~1000 tokens
   - 最近编辑的文件 ~500 tokens
   - import 的文件定义 ~500 tokens

2. **补全缓存**: LRU Cache，key = prefix hash，expire = 30s
   - 相同前缀匹配时直接返回缓存

3. **Debounce**: 350ms debounce + CancellationToken

4. **后处理**:
   - 去重（与已有代码重复的行）

- 括号匹配检查
- 缩进规范化

Continue.dev RAG 实现:
- **向量存储**: LanceDB (Rust-based, 嵌入 WASM)
- **Embedding**: 通过配置的 LLM provider 的 embedding endpoint
- **Chunk 策略**: 按函数/类/代码块分割，每 chunk ~500 tokens
- **索引触发**: 文件保存时增量索引

✅ 可借鉴:
- 上下文 Provider 的插件化架构
- LRU 补全缓存策略
- LanceDB 作为本地向量存储

2.2 Cline/Roo Code 架构

仓库: `github.com/cline/cline` (Apache 2.0)

核心特点:
- 以 Chat + Agent 为核心（不做 inline 补全）
- 使用 Webview 做完整 UI
- 强调 tool use / function calling

架构要点:

代码块

```
cline/
├── src/
│   ├── core/
│   │   ├── Cline.ts              # 核心 Agent 循环
```

```
5   |   |   ├── prompts/              # System prompts
6   |   |   └── tools/                # 工具定义
7   |   |       ├── editFile.ts
8   |   |       ├── readFile.ts
9   |   |       ├── executeCommand.ts
10  |   |       └── browser.ts
11  |   ├── api/
12  |   |   ├── providers/            # API provider 抽象
13  |   |   |   ├── anthropic.ts
14  |   |   |   ├── openai.ts
15  |   |   |   └── openrouter.ts
16  |   |   └── transform/            # 消息格式转换
17  |   ├── integrations/
18  |   |   ├── terminal/             # 终端集成
19  |   |   └── editor/               # 编辑器集成
20  |   └── webview/                  # React UI
```

Cline 的消息协议:

代码块

```
1   // Webview ↔ Extension 消息类型
2   type WebviewMessage =
3     | { type: 'newTask'; text: string; images?: string[] }
4     | { type: 'askResponse'; response: 'yesButtonClicked' | 'noButtonClicked' |
    string }
5     | { type: 'clearTask' }
6     | { type: 'selectImages' }
7     | { type: 'exportCurrentTask' };
8
9   type ExtensionMessage =
10    | { type: 'action'; action: 'chatStarted' | 'chatFinished' }
11    | { type: 'state'; state: ClineState }
12    | { type: 'partialMessage'; partialMessage: ClineMessage }
13    | { type: 'ask'; askType: ClineAskType; text?: string };
```

✅ 可借鉴:

- Webview 消息协议的类型安全设计

- Tool use 的抽象层（为 P2 Agent 模式准备）

- API Provider 的适配器模式

2.3 Cursor 已知技术方案

基于公开信息和逆向分析：

核心技术栈:
- Fork VS Code (Electron) 而非插件形式
- 自定义 Extension Host（深度修改）
- 服务端补全模型：自训练 + GPT-4 + Claude

补全技术:
- **自研补全模型**: 基于 code-specific transformer，针对 FIM 优化
- **Speculative decoding**: 快速本地模型 + 大模型验证
- **上下文窗口**: 全文件 + 跨文件引用图 + 最近编辑历史
- **预测性补全**: 在用户尚未触发时就开始推理

Tab Flow (已知):
- 用户每次按键都会触发请求（aggressive）
- 使用本地 tiny model 做初步筛选
- 大模型补全请求在后台 pre-fetch

CMD+K (Inline Edit):
- 选中代码区域 + 自然语言指令
- 使用 Diff 视图展示修改（绿/红高亮）
- 支持 Accept/Reject 每个 hunk

⚠ 对 LinkCode 的启示:
- 作为插件，无法做到 Cursor 的深度集成

- 补全延迟是最大挑战（Cursor fork VS Code 省去了 IPC 开销）
- Ghost Text 体验受限于 VS Code API

2.4 Copilot 已知技术方案

技术架构:
- VS Code Extension + GitHub Copilot Language Server
- 使用 LSP 的自定义扩展协议
- 后端 Codex/GPT 模型

补全实现:
- **Ghosttext**: 使用 `InlineCompletionItemProvider`（标准 API）
- **上下文**: 当前文件 + 打开的 tabs（按相关性排序）+ import graph
- **Prompt 格式**:
  ```
  // Path: src/utils/auth.ts
  // Compare this snippet from src/models/user.ts:
  // export interface User { id: string; name: string; }

  [当前文件的代码上下文]
  [光标位置]
  ```
- **后处理**: 括号匹配、缩进修正、重复行去除
- **Telemetry**: 大量的 accept/reject 数据收集

Copilot Chat:
- 使用 VS Code Chat API (`vscode.chat.createChatParticipant`)
- @workspace：使用 VS Code 内置的 workspace indexing

- Streaming via SSE

✅ 可借鉴:
- "Compare this snippet from" 的上下文引用格式
- Tab 按键精排序的 prompt 策略
- 基于 accept rate 的模型选择优化

---

3. 多模型路由引擎详细设计

3.1 任务分类器方案

推荐方案: 规则引擎（MVP）+ LLM 辅助分类（P1）

方案对比:

方案	延迟	准确率	复杂度	推荐阶段
规则引擎	<1ms	~85%	低	✅ MVP
ML 分类器 (本地)	<10ms	~92%	中	P1
LLM 判断	200-500ms	~97%	低	❌ 太慢
混合(规则+ML)	<10ms	~95%	中	✅ P1

规则引擎实现（MVP）:

代码块

```
// types.ts
enum TaskType {
  INLINE_COMPLETION = 'inline_completion',     // 行内补全
```

```
4     BLOCK_COMPLETION = 'block_completion',        // 块补全
5     CHAT_SIMPLE = 'chat_simple',                  // 简单问答
6     CHAT_COMPLEX = 'chat_complex',                // 复杂架构/推理
7     CODE_REVIEW = 'code_review',                  // 代码审查
8     CODE_EXPLAIN = 'code_explain',                // 代码解释
9     DOC_GENERATION = 'doc_generation',            // 文档/注释生成
10    TEST_GENERATION = 'test_generation',          // 测试生成
11    REFACTOR = 'refactor',                        // 重构
12    SQL_REGEX = 'sql_regex',                      // SQL/正则
13  }
14
15  // classifier.ts
16  interface ClassificationResult {
17    taskType: TaskType;
18    confidence: number;
19    estimatedTokens: number;
20    reasoning: string;
21  }
22
23  class RuleBasedClassifier {
24    classify(context: TaskContext): ClassificationResult {
25      // 1. 补全类任务（来自 InlineCompletionProvider）
26      if (context.source === 'completion') {
27        const prefixLines = context.prefix.split('\n');
28        const lastLine = prefixLines[prefixLines.length - 1].trim();
29
30        // 行内补全：当前行有内容
31        if (lastLine.length > 0) {
32          return {
33            taskType: TaskType.INLINE_COMPLETION,
34            confidence: 0.95,
35            estimatedTokens: 50,
36            reasoning: 'Inline completion detected from typing context'
37          };
38        }
39
40        // 块补全：空行/新函数开始
41        return {
42          taskType: TaskType.BLOCK_COMPLETION,
43          confidence: 0.9,
44          estimatedTokens: 200,
45          reasoning: 'Block completion - empty line or new block'
46        };
47      }
48
49      // 2. Chat 类任务
50      const message = context.userMessage.toLowerCase();
```

```

    // 代码审查关键词
    if (this.matchPatterns(message, [
      /review|审查|code review|cr|检查代码|bug.*find|找.*bug/i
    ])) {
      return {
        taskType: TaskType.CODE_REVIEW,
        confidence: 0.9,
        estimatedTokens: this.estimateTokens(context),
        reasoning: 'Code review keywords detected'
      };
    }

    // 测试生成
    if (this.matchPatterns(message, [
      /tes
```

代码块

```
t|测试|单测|unit test|spec|jest|mocha|pytest/i
    ])) {
      return {
        taskType: TaskType.TEST_GENERATION,
        confidence: 0.9,
        estimatedTokens: this.estimateTokens(context),
        reasoning: 'Test generation keywords detected'
      };
    }

    // 文档/注释
    if (this.matchPatterns(message, [
      /文档|注释|comment|doc|jsdoc|tsdoc|docstring|readme/i
    ])) {
      return {
        taskType: TaskType.DOC_GENERATION,
        confidence: 0.85,
        estimatedTokens: this.estimateTokens(context),
        reasoning: 'Documentation generation keywords detected'
      };
    }

    // SQL/正则
    if (this.matchPatterns(message, [
      /sql|query|select.*from|正则|regex|regexp|匹配/i
    ])) {
      return {
```

```
28          taskType: TaskType.SQL_REGEX,
29          confidence: 0.85,
30          estimatedTokens: this.estimateTokens(context),
31          reasoning: 'SQL/Regex task detected'
32        };
33      }
34
35      // 复杂架构（长消息 + 架构关键词）
36      if (context.userMessage.length > 200 || this.matchPatterns(message, [
37        /架构|设计|design|architecture|系统|方案|实现.*方案|how.*implement/i
38      ])) {
39        return {
40          taskType: TaskType.CHAT_COMPLEX,
41          confidence: 0.75,
42          estimatedTokens: this.estimateTokens(context),
43          reasoning: 'Complex architecture/design question'
44        };
45      }
46
47      // 默认：简单问答
48      return {
49        taskType: TaskType.CHAT_SIMPLE,
50        confidence: 0.7,
51        estimatedTokens: this.estimateTokens(context),
52        reasoning: 'Default classification: simple chat'
53      };
54    }
55
56    private matchPatterns(text: string, patterns: RegExp[]): boolean {
57      return patterns.some(p => p.test(text));
58    }
59
60    private estimateTokens(context: TaskContext): number {
61      // 粗略估计：1 Chinese char ≈ 2 tokens, 1 English word ≈ 1.3 tokens
62      const inputLength = (context.userMessage?.length || 0) +
    (context.codeContext?.length || 0);
63      return Math.ceil(inputLength * 1.5);
64    }
65  }
```

3.2 路由决策数据结构

```typescript
// route-config.ts
interface ModelConfig {
  modelId: string;           // Smoothlink API 的 model ID
  displayName: string;       // 展示名称
  provider: string;          // 提供商 (deepseek, anthropic, qwen)
  maxTokens: number;         // 最大 token 数
  inputPrice: number;        // 输入价格 ($/M tokens)
  outputPrice: number;       // 输出价格 ($/M tokens)
  supportsStreaming: boolean;
  supportsFIM: boolean;      // 是否支持 Fill-in-the-Middle
  averageLatency: number;    // 平均延迟 ms (历史统计)
}

interface RouteRule {
  taskType: TaskType;
  primary: string;           // 主模型 ID
  fallback: string[];        // 备选模型列表 (按优先级)
  maxRetries: number;
  timeoutMs: number;
  conditions?: {             // 可选条件
    maxInputTokens?: number; // 超过此值切换到更大模型
    userPlan?: string[];     // 仅某些计划可用
    languages?: string[];    // 仅某些编程语言
  };
}

// 默认路由配置
const DEFAULT_ROUTE_CONFIG: Record<TaskType, RouteRule> = {
  [TaskType.INLINE_COMPLETION]: {
    taskType: TaskType.INLINE_COMPLETION,
    primary: 'deepseek-coder-v2-lite',
    fallback: ['deepseek-coder-v2', 'qwen2.5-coder-7b'],
    maxRetries: 1,
    timeoutMs: 2000,
  },
  [TaskType.BLOCK_COMPLETION]: {
    taskType: TaskType.BLOCK_COMPLETION,
    primary: 'deepseek-coder-v2',
    fallback: ['deepseek-coder-v2-lite', 'qwen2.5-coder-32b'],
    maxRetries: 1,
    timeoutMs: 5000,
  },
  [TaskType.CHAT_SIMPLE]: {
    taskType: TaskType.CHAT_SIMPLE,
    primary: 'deepseek-v3',
    fallback: ['qwen2.5-72b', 'glm-4'],
    maxRetries: 2,
```

```
48      timeoutMs: 30000,
49    },
50    [TaskType.CHAT_COMPLEX]: {
51      taskType: TaskType.CHAT_COMPLEX,
52      primary: 'claude-sonnet-4-20250514',
53      fallback: ['deepseek-v3', 'gpt-4o'],
54      maxRetries: 2,
55      timeoutMs: 60000,
56      conditions: {
57        userPlan: ['pro', 'team', 'enterprise'],
58      },
59    },
60    [TaskType.CODE_REVIEW]: {
61      taskType: TaskType.CODE_REVIEW,
62      primary: 'claude-sonnet-4-20250514',
63      fallback: ['deepseek-v3', 'gpt-4o'],
64      maxRetries: 2,
65      timeoutMs: 60000,
66    },
67    [TaskType.TEST_GENERATION]: {
68      task
```

代码块

```
1   Type: TaskType.TEST_GENERATION,
2       primary: 'qwen2.5-coder-32b',
3       fallback: ['deepseek-coder-v2', 'deepseek-v3'],
4       maxRetries: 2,
5       timeoutMs: 30000,
6     },
7     [TaskType.DOC_GENERATION]: {
8       taskType: TaskType.DOC_GENERATION,
9       primary: 'deepseek-coder-v2',
10      fallback: ['qwen2.5-coder-32b', 'deepseek-v3'],
11      maxRetries: 2,
12      timeoutMs: 15000,
13    },
14    [TaskType.SQL_REGEX]: {
15      taskType: TaskType.SQL_REGEX,
16      primary: 'deepseek-v3',
17      fallback: ['qwen2.5-72b', 'deepseek-coder-v2'],
18      maxRetries: 2,
19      timeoutMs: 15000,
20    },
21  };
```

3.3 Fallback 策略详细流程

代码块

```
class RoutingEngine {
  private config: Record<TaskType, RouteRule>;
  private modelRegistry: Map<string, ModelConfig>;
  private routeLog: RouteLogEntry[] = [];

  async route(context: TaskContext): Promise<RouteResult> {
    const startTime = Date.now();

    // Step 1: 分类
    const classification = this.classifier.classify(context);

    // Step 2: 获取路由规则
    const rule = this.config[classification.taskType];

    // Step 3: 检查用户计划权限
    if (rule.conditions?.userPlan &&
        !rule.conditions.userPlan.includes(context.userPlan)) {
      // 降级到 free 可用的模型
      return this.routeForFreePlan(classification);
    }

    // Step 4: 尝试主模型
    const models = [rule.primary, ...rule.fallback];
    let lastError: Error | null = null;

    for (let i = 0; i < models.length; i++) {
      const modelId = models[i];
      const model = this.modelRegistry.get(modelId);
      if (!model) continue;

      // 检查模型健康状态
      if (this.circuitBreaker.isOpen(modelId)) {
        this.logRoute({
          modelId,
          status: 'circuit_open',
          reason: 'Circuit breaker open, skipping',
        });
        continue;
      }

```

```
41        try {
42          const result = await this.callModel(modelId, context, rule.timeoutMs);
43
44          // 成功
45          this.logRoute({
46            requestId: result.requestId,
47            taskType: classification.taskType,
48            modelId,
49            isPrimary: i === 0,
50            latencyMs: Date.now() - startTime,
51            inputTokens: result.usage.promptTokens,
52            outputTokens: result.usage.completionTokens,
53            status: 'success',
54            cost: this.calculateCost(model, result.usage),
55          });
56
57          return result;
58
59        } catch (error: any) {
60          lastError = error;
61
62          this.logRoute({
63            taskType: classification.taskType,
64            modelId,
65            isPrimary: i === 0,
66            latencyMs: Date.now() - startTime,
67            status: 'error',
68            erro
```

代码块

```
1   r: error.message,
2           });
3
4           // 更新熔断器
5           this.circuitBreaker.recordFailure(modelId);
6
7           // 判断是否可重试
8           if (!this.isRetryableError(error)) {
9             // 非重试错误（如 auth 失败），直接报错
10            throw error;
11          }
12
13          // 继续尝试下一个模型
14          continue;
15        }
```

```
    }

    // 所有模型都失败
    throw new Error(`All models failed for ${classification.taskType}: ${lastError?.message}`);
  }

  private isRetryableError(error: any): boolean {
    // 429: Rate limited
    // 500, 502, 503, 504: Server errors
    const retryableCodes = [429, 500, 502, 503, 504];
    return retryableCodes.includes(error.status);
  }
}

// 熔断器实现
class CircuitBreaker {
  private failures: Map<string, { count: number; lastFailure: number }> = new Map();
  private readonly threshold = 5;        // 5 次失败触发
  private readonly resetTimeout = 60000; // 60 秒后重置

  isOpen(modelId: string): boolean {
    const record = this.failures.get(modelId);
    if (!record) return false;
    if (record.count < this.threshold) return false;
    if (Date.now() - record.lastFailure > this.resetTimeout) {
      this.failures.delete(modelId);
      return false;
    }
    return true;
  }

  recordFailure(modelId: string): void {
    const record = this.failures.get(modelId) || { count: 0, lastFailure: 0 };
    record.count++;
    record.lastFailure = Date.now();
    this.failures.set(modelId, record);
  }

  recordSuccess(modelId: string): void {
    this.failures.delete(modelId);
  }
}
```

3.4 用户自定义路由

代码块

```
// settings.json (VS Code)
{
  "linkcode.routing.customRules": {
    "inline_completion": {
      "primary": "deepseek-coder-v2",
      "fallback": ["qwen2.5-coder-32b"]
    },
    "chat_complex": {
      "primary": "gpt-4o",
      "fallback": ["claude-sonnet-4-20250514"]
    }
  },
  "linkcode.routing.preferCheap": true,
  "linkcode.routing.maxLatencyMs": 3000
}
```

3.5 路由日志和透明度

代码块

```
// route-log.ts
interface RouteLogEntry {
  requestId: string;
  timestamp: number;
  taskType: TaskType;
  modelId: string;
  isPrimary: boolean;
  latencyMs: number;
  inputTokens?: number;
  outputTokens?: number;
  cost?: number;            // 美元
  status: 'success' | 'error' | 'circuit_open' | 'timeout';
  error?: string;
  reasoning?: string;       // 为什么选择这个模型
}

// 在 Chat UI 中展示
```

```
// 每条消息底部小字："⚡ deepseek-v3 · 1.2s · 340 tokens · ¥0.003"
// 点击展开：完整路由决策过程

// 在设置页提供路由日志查看器
// WebView 中展示最近 100 条路由记录的表格
```

---

4. 代码补全的完整技术方案

4.1 FIM (Fill-in-the-Middle) 协议

FIM 是代码补全模型的标准输入格式，让模型在前缀和后缀之间生成代码。

标准 FIM 格式（DeepSeek-Coder）:

代码块

```
<｜fim_begin｜>{prefix}<｜fim_hole｜>{suffix}<｜fim_end｜>
```

不同模型的 FIM Token:

模型	Prefix Token	Suffix Token	Middle Token
DeepSeek-Coder	`<｜fim_begin｜>`	`<｜fim_hole｜>`	`<｜fim_end｜>`
Qwen2.5-Coder	`<|fim_prefix|>`	`<|fim_suffix|>`	`<|fim_middle|>`
StarCoder	`<fim_prefix>`	`<fim_suffix>`	`<fim_middle>`
CodeLlama	`<PRE>`	`<SUF>`	`<MID>`

FIM Prompt 构建:

代码块

```
class FIMPromptBuilder {
  buildPrompt(
    prefix: string,
    suffix: string,
    model: string,
    options: FIMOptions
  ): string {
    const tokens = this.getModelTokens(model);

    // 截断以适应上下文窗口
    const maxPrefixTokens = Math.floor(options.maxContextTokens * 0.6);
    const maxSuffixTokens = Math.floor(options.maxContextTokens * 0.3);

    const trimmedPrefix = this.trimToTokenLimit(prefix, maxPrefixTokens, 'end');
    const trimmedSuffix = this.trimToTokenLimit(suffix, maxSuffixTokens, 'start');

    return `${tokens.prefix}${trimmedPrefix}${tokens.suffix}${trimmedSuffix}${tokens.middle}`;
  }

  private getModelTokens(model: string): FIMTokens {
    const tokenMap: Record<string, FIMTokens> = {
      'deepseek-coder': {
        prefix: '<｜fim_begin｜>',
        suffix: '<｜fim_hole｜>',
        middle: '<｜fim_end｜>'
      },
      'qwen2.5-coder': {
        prefix: '<|fim_prefix|>',
        suffix: '<|fim_suffix|>',
        middle: '<|fim_middle|>'
      },
    };
    return tokenMap[model] || tokenMap['deepseek-coder'];
  }
}
```

如果 Smoothlink 使用 OpenAI 兼容 API:

```
// 某些模型通过 OpenAI 兼容接口支持 FIM
const response = await fetch('https://api.smoothlink.ai/v1/completions', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${apiKey}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    model: 'deepseek-coder-v2-lite',
    prompt: fimPrompt,    // FIM 格式的 prompt
    max_tokens: 128,
    temperature: 0.0,     // 补全用低温度
    stop: ['\n\n', '<｜fim_end｜>', '<|endoftext|>'],
    stream: true
  })
});
```

4.2 上下文窗口构建

代码块

```
interface CompletionContext {
  // 核心上下文
  prefix: string;          // 光标前代码
  suffix: string;          // 光标后代码

  // 扩展上下文
  imports: string;         // 当前文件的 import 语句
  relatedSnippets: string[]; // 相关文件的代码片段

  // 元信息
  language: string;
  filePath: string;
  cursorLine: number;
  cursorColumn: number;
}

class ContextBuilder {
  // 上下文预算分配（以 token 为单位）
  private readonly BUDGET = {
    prefix: 2048,          // 光标前: ~60%
    suffix: 512,           // 光标后: ~15%
    imports: 256,          // import 语句: ~8%
```

```
    relatedFiles: 512,     // 相关文件: ~15%
    metadata: 64,          // 文件路径等元信息: ~2%
  };

  async buildContext(
    document: vscode.TextDocument,
    position: vscode.Position
  ): Promise<CompletionContext> {
    // 1. 获取光标前后代码
    const fullText = document.getText();
    const offset = document.offsetAt(position);

    let prefix = fullText.substring(0, offset);
    let suffix = fullText.substring(offset);

    // 截断到 token 预算
    prefix = this.trimFromStart(prefix, this.BUDGET.prefix);
    suffix = this.trimFromEnd(suffix, this.BUDGET.suffix);

    // 2. 提取 import 语句
    const imports = this.extractImports(document);

    // 3. 获取相关文件片段
    const relatedSnippets = await this.getRelatedSnippets(document, position);

    // 4. 构建最终 prompt (含文件路径注释)
    return {
      prefix: `// Path: ${vscode.workspace.asRelativePath(document.uri)}\n${imports}\n${prefix}`,
      suffix,
      imports,
      relatedSnippets,
      language: document.languageId,
      filePath: document.uri.fsPath,
      cursorLine: position.line,
      cursorColumn: position.character,
    };
  }

  // 相关文件选取策略
  private async getRelatedSnippets(
    document: vscode.TextDocument,
    position: vscode.Position
  ): Promise<string[]> {
    const snippets: string[] = [];

    // 策略 1: 打开的 Tab 中同类型文件 (按最近访问排序)
```

```
      const openDocs = vscode.workspace.textDocuments
        .filter(d => d.languageId === document.languageId && d.uri.toString() !== document.uri.toString())

```

代码块

```
    .slice(0, 3); // 最多 3 个

      for (const doc of openDocs) {
        snippets.push(
          `// From: ${vscode.workspace.asRelativePath(doc.uri)}\n` +
          this.trimFromEnd(doc.getText(), 200) // 每个文件最多 200 tokens
        );
      }

      // 策略 2: import 引用的文件
      const importPaths = this.resolveImportPaths(document);
      for (const importPath of importPaths.slice(0, 2)) {
        try {
          const doc = await vscode.workspace.openTextDocument(importPath);
          // 只取 export 的符号定义
          const exports = this.extractExports(doc);
          if (exports) {
            snippets.push(`// From: ${vscode.workspace.asRelativePath(importPath)}\n${exports}`);
          }
        } catch {}
      }

      return snippets;
    }

    private extractImports(document: vscode.TextDocument): string {
      const lines = document.getText().split('\n');
      const importLines = lines.filter(line => {
        const trimmed = line.trim();
        return trimmed.startsWith('import ') ||
               trimmed.startsWith('from ') ||
               trimmed.match(/^(const|let|var)\s+.*=\s*require\(/);
      });
      return importLines.join('\n');
    }
  }
```

4.3 Debounce 和 Cancel 策略

代码块

```
class CompletionManager {
  private debounceTimer: NodeJS.Timeout | null = null;
  private currentAbortController: AbortController | null = null;
  private lastRequestId = 0;

  // Debounce 配置
  private readonly DEBOUNCE_MS = 200;        // 打字后 200ms 才触发
  private readonly FAST_DEBOUNCE_MS = 75;     // 行尾/空行时更快

  async provideCompletion(
    document: vscode.TextDocument,
    position: vscode.Position,
    token: vscode.CancellationToken
  ): Promise<vscode.InlineCompletionItem[] | undefined> {

    // 1. 判断是否需要快速响应
    const line = document.lineAt(position.line).text;
    const isEndOfLine = position.character >= line.trimEnd().length;
    const debounceMs = isEndOfLine ? this.FAST_DEBOUNCE_MS : this.DEBOUNCE_MS;

    // 2. Cancel 之前的请求
    if (this.currentAbortController) {
      this.currentAbortController.abort();
    }

    // 3. Debounce
    const requestId = ++this.lastRequestId;
    await this.delay(debounceMs, token);

    // 4. 检查是否已被取消
    if (token.isCancellationRequested || requestId !== this.lastRequestId) {
      return undefined;
    }

    // 5. 发起请求
    this.currentAbortController = new AbortController();

    try {
      const context = await this.contextBuilder.buildContext(document, position);
```

```
40        const result = await this.apiClient.complete(context, {
41          signal: this.currentAbortController.signal,
42          timeout: 3000, // 3 秒超时
43        });
44
45        // 6. 请求期间用户可能已继续输入
46        if (token.isCancellationRequested || requestId !== this.lastRequestId) {
47          return undefined;
48        }
49
50        return this.formatResult(result, position);
51
52      } catch (error: any) {
53        if (error.name === 'AbortError') return undefined;
54        console.error('Completion error:', error);
55        return undefined;
56      }
57    }
58
59    private delay(ms: number, token: vscode.CancellationToken): Promise<void> {
60      return new Promise((resolve, reject) => {
61        const timer = setTimeout(resolve, ms);
62        token.onCancellationReques
```

代码块

```
1   ted(() => {
2           clearTimeout(timer);
3           reject(new Error('Cancelled'));
4         });
5       });
6     }
7   }
```

4.4 Ghost Text 渲染

Ghost Text 由 VS Code 原生处理，通过 `InlineCompletionItem` 控制：

代码块

```
1   function formatResult(
```

```
    completion: string,
    position: vscode.Position
  ): vscode.InlineCompletionItem[] {
    // 1. 清理补全文本
    let text = completion;

    // 去除尾部多余换行
    text = text.replace(/\n{3,}/g, '\n\n');

    // 去除与已有代码重复的部分
    text = this.deduplicateWithExisting(text, position);

    // 2. 创建 InlineCompletionItem
    const item = new vscode.InlineCompletionItem(
      text,
      new vscode.Range(position, position)
    );

    // 3. 设置过滤文本（用于前缀匹配）
    // 当用户继续输入时，VS Code 用这个来判断是否仍然展示
    item.filterText = text;

    // 4. 设置命令（用于追踪接受/拒绝）
    item.command = {
      command: 'linkcode.completionAccepted',
      title: 'Completion Accepted',
      arguments: [{ text, model: 'deepseek-coder-v2-lite', latencyMs: 300 }]
    };

    return [item];
  }
```

⚠ 限制:

- 无法自定义 Ghost Text 的颜色/样式
- 无法做 diff 高亮（Cursor 可以因为 fork 了 VS Code）
- 多行 Ghost Text 在某些主题下可能不够清晰

4.5 多行补全 vs 单行补全

```
class CompletionDecisionMaker {
  shouldMultiLine(context: CompletionContext): boolean {
    const lastLine = context.prefix.split('\n').pop()?.trim() || '';

    // 多行补全场景
    const multiLineIndicators = [
      // 函数定义开始
      /^(function|def|fn|func)\s/,
      /^(async\s+)?function/,
      // 箭头函数
      /=>\s*\{?\s*$/,
      // 类/接口定义
      /^(class|interface|struct|enum|type)\s/,
      // 条件/循环
      /^(if|for|while|switch|match)\s/,
      // 空行（新代码块）
      /^\s*$/,
      // 只有左括号
      /\{\s*$/,
      // JSX/TSX 开标签
      /<\w+[^/]*>\s*$/,
    ];

    return multiLineIndicators.some(regex => regex.test(lastLine));
  }

  getMaxTokens(context: CompletionContext): number {
    if (this.shouldMultiLine(context)) {
      return 256;  // 多行最多 256 tokens
    }
    return 64;     // 单行最多 64 tokens
  }

  getStopSequences(context: CompletionContext): string[] {
    if (this.shouldMultiLine(context)) {
      return ['\n\n\n'];  // 两个空行停止
    }
    return ['\n'];          // 单行在换行符停止
  }
}
```

4.6 缓存策略

代码块

```
class CompletionCache {
  private cache: Map<string, CacheEntry> = new Map();
  private readonly MAX_SIZE = 100;
  private readonly TTL_MS = 30_000; // 30 秒过期

  // 缓存 key = 文件 URI + 前缀 hash
  private getCacheKey(uri: string, prefix: string): string {
    // 只用最后 500 字符做 hash，避免计算开销
    const relevantPrefix = prefix.slice(-500);
    const hash = this.simpleHash(relevantPrefix);
    return `${uri}:${hash}`;
  }

  get(uri: string, prefix: string): string | null {
    const key = this.getCacheKey(uri, prefix);
    const entry = this.cache.get(key);

    if (!entry) return null;
    if (Date.now() - entry.timestamp > this.TTL_MS) {
      this.cache.delete(key);
      return null;
    }

    return entry.completion;
  }

  // 前缀命中：用户继续输入了几个字符，之前的补全可能仍然有效
  getWithPrefixMatch(uri: string, prefix: string): string | null {
    for (const [key, entry] of this.cache) {
      if (!key.startsWith(uri)) continue;
      if (Date.now() - entry.timestamp > this.TTL_MS) continue;

      // 检查是否是前缀关系
      if (prefix.startsWith(entry.originalPrefix)) {
        const extraTyped = prefix.slice(entry.originalPrefix.length);
        if (entry.completion.startsWith(extraTyped)) {
          // 用户输入匹配了补全的开头，返回剩余部分
          return entry.completion.slice(extraTyped.length);
        }
      }
    }
    return null;
  }

  set(uri: string, prefix: string, completion: string): void {
    if (this.cache.size >= this.MAX_SIZE) {
      // LRU 淘汰
```

```
      const oldestKey = this.cache.keys().next().value;
      if (oldestKey) this.cache.delete(oldestKey);
    }

    const key = this.getCacheKey(uri, prefix);
    this.cache.set(key, {
      completion,
      originalPrefix: prefix,
      timestamp: Date.now(),
    });
  }

  private simpleHash(str: string): string {
    let hash = 0;
    for (let i = 0; i < str.length; i++) {
      const char = str.charCodeAt(i);
      hash = ((hash << 5) - hash) + char;
      hash = hash & hash; // 转为 32bit
    }
    return hash.toString(36
```

代码块

```
);
  }
}

interface CacheEntry {
  completion: string;
  originalPrefix: string;
  timestamp: number;
}
```

---

5. Chat 面板完整技术方案

5.1 Webview 与 Extension Host 消息协议

代码块

```
// shared/messages.ts - 两端共享的消息类型定义

// Extension → Webview 的消息
type ExtensionToWebviewMessage =
  | { type: 'streamStart'; messageId: string; model: string }
  | { type: 'streamChunk'; messageId: string; content: string }
  | { type: 'streamEnd'; messageId: string; usage: TokenUsage }
  | { type: 'streamError'; messageId: string; error: string }
  | { type: 'updateBalance'; balance: number; currency: string }
  | { type: 'routeInfo'; messageId: string; route: RouteLogEntry }
  | { type: 'fileList'; files: FileItem[] }
  | { type: 'symbolList'; symbols: SymbolItem[] }
  | { type: 'historyLoaded'; conversations: Conversation[] }
  | { type: 'settingsUpdate'; settings: UserSettings }
  | { type: 'themeChanged'; theme: 'light' | 'dark' | 'high-contrast' };

// Webview → Extension 的消息
type WebviewToExtensionMessage =
  | { type: 'sendMessage'; text: string; attachments: Attachment[]; conversationId?: string }
  | { type: 'cancelGeneration'; messageId: string }
  | { type: 'selectModel'; modelId: string; scope: 'completion' | 'chat' }
  | { type: 'insertCode'; code: string; filePath?: string }
  | { type: 'copyCode'; code: string }
  | { type: 'openFile'; filePath: string; line?: number }
  | { type: 'searchFiles'; query: string }  // @file 自动补全
  | { type: 'searchSymbols'; query: string } // @symbol 自动补全
  | { type: 'loadHistory'; offset: number; limit: number }
  | { type: 'deleteConversation'; conversationId: string }
  | { type: 'updateSettings'; settings: Partial<UserSettings> }
  | { type: 'requestBalance' }
  | { type: 'webviewReady' };

interface Attachment {
  type: 'file' | 'selection' | 'workspace';
  content: string;
  filePath?: string;
  startLine?: number;
  endLine?: number;
}

interface TokenUsage {
  promptTokens: number;
  completionTokens: number;
  totalTokens: number;
  cost: number;
}
```

消息处理架构:

代码块

```
// Extension Host 端
class ChatMessageHandler {
  constructor(
    private webviewView: vscode.WebviewView,
    private routingEngine: RoutingEngine,
    private apiClient: SmoothlinkClient,
  ) {
    this.webviewView.webview.onDidReceiveMessage(this.handleMessage.bind(this));
  }

  private async handleMessage(message: WebviewToExtensionMessage) {
    switch (message.type) {
      case 'sendMessage':
        await this.handleSendMessage(message);
        break;
      case 'cancelGeneration':
        this.cancelGeneration(message.messageId);
        break;
      case 'insertCode':
        await this.insertCodeToEditor(message.code, message.filePath);
        break;
      case 'searchFiles':
        await this.handleFileSearch(message.query);
        break;
      // ...
    }
  }

  private async handleSendMessage(message: { text: string; attachments: Attachment[] }) {
    const messageId = crypto.randomUUID();

    // 通知 Webview 开始 streaming
    this.postMessage({ type: 'streamStart', messageId, model: '' });

    try {
      // 构建完整 prompt（含 attachments 上下文）
      const fullContext = await this.buildChatContext(message);
```

```

        // 路由决策
        const route = await this.routingEngine.route({
          source: 'chat',
          userMessage: message.text,
          codeContext: fullContext,
          userPlan: await this.getUserPlan(),
        });

        // 通知选择的模型
        this.postMessage({ type: 'routeInfo', messageId, route: route.logEntry
});

        // 流式调用
        const stream = await this.apiClient.chatStream({
          model: route.modelId,
          messages: this.buildMessages(fullContext),
          stream: true,
        });

        for await (const chunk of stream) {
          if (this.isCancelled(messageId)) break;

          this.postMessage({
            type: 'streamChunk',
            messageId,
            content: chunk.content,
          });
        }

        this.postMessage({
          type: 'streamEnd',
          messageId,
          usage: st
```

代码块

```
ream.usage,
        });

      } catch (error: any) {
        this.postMessage({
          type: 'streamError',
          messageId,
          error: error.message,
        });
```

```
      }
    }

    private postMessage(message: ExtensionToWebviewMessage) {
      this.webviewView.webview.postMessage(message);
    }
  }
```

5.2 Streaming 响应实现

Extension Host 端（Node.js，处理 SSE）:

代码块

```
class SmoothlinkClient {
  async *chatStream(params: ChatParams): AsyncGenerator<StreamChunk> {
    const response = await fetch(`${this.baseUrl}/v1/chat/completions`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${this.apiKey}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        ...params,
        stream: true,
      }),
      signal: params.signal,
    });

    if (!response.ok) {
      throw new APIError(response.status, await response.text());
    }

    const reader = response.body!.getReader();
    const decoder = new TextDecoder();
    let buffer = '';
    let totalUsage: TokenUsage = { promptTokens: 0, completionTokens: 0, totalTokens: 0, cost: 0 };

    while (true) {
      const { done, value } = await reader.read();
      if (done) break;
```

```

        buffer += decoder.decode(value, { stream: true });

        // 解析 SSE
        const lines = buffer.split('\n');
        buffer = lines.pop() || ''; // 最后一行可能不完整

        for (const line of lines) {
          if (line.startsWith('data: ')) {
            const data = line.slice(6).trim();
            if (data === '[DONE]') {
              return;
            }

            try {
              const parsed = JSON.parse(data);
              const content = parsed.choices?.[0]?.delta?.content;

              if (content) {
                yield { content, done: false };
              }

              // 收集 usage 信息
              if (parsed.usage) {
                totalUsage = parsed.usage;
              }
            } catch {}
          }
        }
      }

      // 返回最终 usage
      yield { content: '', done: true, usage: totalUsage };
    }
  }
```

Webview 端（React，接收消息并渲染）:

代码块

```
// React Hook for streaming messages
function useStreamingMessages() {
  const [messages, setMessages] = useState<ChatMessage[]>([]);
```

```
4     const streamBufferRef = useRef<Map<string, string>>(new Map());
5
6     useEffect(() => {
7       const handler = (event: MessageEvent) => {
8         const message = event.data as ExtensionToWebviewMessage;
9
10        switch (message.type) {
11          case 'streamStart': {
12            setMessages(prev => [...prev, {
13              id: message.messageId,
14              role: 'assistant',
15              content: '',
16              model: message.model,
17              status: 'streaming',
18            }]);
19            streamBufferRef.current.set(message.messageId, '');
20            break;
21          }
22
23          case 'streamChunk': {
24            // 累积到 buffer 然后批量更新（减少 React render）
25            const buffer = streamBufferRef.current.get(message.messageId) || '';
26            streamBufferRef.current.set(message.messageId, buffer +
    message.content);
27
28            // 使用 requestAnimationFrame 批量更新
29            requestAnimationFrame(() => {
30              const content = streamBufferRef.current.get(message.messageId) ||
    '';
31              setMessages(prev => prev.map(m =>
32                m.id === message.messageId
33                  ? { ...m, content, status: 'streaming' as const }
34                  : m
35              ));
36            });
37            break;
38          }
39
40          case 'streamEnd': {
41            setMessages(prev => prev.map(m =>
42              m.id === message.messageId
43                ? { ...m, status: 'complete' as const, usage: message.usage }
44                : m
45            ));
46            streamBufferRef.current.delete(message.messageId);
47            break;
48          }
```

```

        case 'streamError': {
          setMessages(prev => prev.map(m =>
            m.id === message.messageId
              ? { ...m, status: 'error' as const, error: message.error }
              : m
          ));
          break;
        }
      }
    };

    window.addEventListener('message', handler);
```

代码块

```

    return () => window.removeEventListener('message', handler);
  }, []);

  return messages;
}
```

5.3 @file/@workspace/@symbol 实现

代码块

```
// Extension Host 端

// @file 实现
class FileContextProvider {
  async searchFiles(query: string): Promise<FileItem[]> {
    // 使用 VS Code 的文件搜索 API
    const files = await vscode.workspace.findFiles(
      `**/*${query}*`,        // glob pattern
      '**/node_modules/**',    // 排除
      20                       // 最多 20 个
    );

    return files.map(uri => ({
      path: vscode.workspace.asRelativePath(uri),
      name: path.basename(uri.fsPath),
```

```
      uri: uri.toString(),
    }));
  }

  async getFileContent(uri: string): Promise<string> {
    const document = await vscode.workspace.openTextDocument(vscode.Uri.parse(uri));
    return document.getText();
  }
}

// @workspace 实现
class WorkspaceContextProvider {
  async getWorkspaceContext(query: string): Promise<string> {
    // P0: 简单实现 - 搜索文件内容
    const results = await vscode.workspace.findFiles('**/*.{ts,js,py,java,go}', '**/node_modules/**', 50);

    const relevant: string[] = [];
    for (const uri of results) {
      const doc = await vscode.workspace.openTextDocument(uri);
      const text = doc.getText();
      if (text.toLowerCase().includes(query.toLowerCase())) {
        relevant.push(`// File: ${vscode.workspace.asRelativePath(uri)}\n${text.slice(0, 500)}`);
      }
    }

    return relevant.join('\n\n---\n\n');
  }
}

// @symbol 实现
class SymbolContextProvider {
  async searchSymbols(query: string): Promise<SymbolItem[]> {
    const symbols = await vscode.commands.executeCommand<vscode.SymbolInformation[]>(
      'vscode.executeWorkspaceSymbolProvider',
      query
    );

    return (symbols || []).slice(0, 20).map(s => ({
      name: s.name,
      kind: vscode.SymbolKind[s.kind],
      path: vscode.workspace.asRelativePath(s.location.uri),
      line: s.location.range.start.line,
    }));
```

```
    }
  }
```

Webview 端 @mention 输入框:

代码块

```
// React component for @mention input
function ChatInput({ onSend }: { onSend: (text: string, attachments: Attachment[]) => void }) {
  const [text, setText] = useState('');
  const [mentionQuery, setMentionQuery] = useState<{ type: string; query: string } | null>(null);
  const [suggestions, setSuggestions] = useState<any[]>([]);

  const handleInput = (value: string) => {
    setText(value);

    // 检测 @ 触发
    const match = value.match(/@(file|workspace|symbol)\s+(\S*)$/);
    if (match) {
      setMentionQuery({ type: match[1], query: match[2] });
      vscode.postMessage({
        type: match[1] === 'file' ? 'searchFiles' : 'searchSymbols',
        query: match[2]
      });
    } else {
      setMentionQuery(null);
    }
  };

  return (
    <div className="chat-input-container">
      <textarea
        value={text}
        onChange={e => handleInput(e.target.value)}
        placeholder="输入消息... 使用 @file @workspace @symbol 引用代码"
        onKeyDown={e => {
          if (e.key === 'Enter' && !e.shiftKey) {
            e.preventDefault();
            onSend(text, []);
            setText('');
          }
```

```
35          }}
36        />
37        {mentionQuery && suggestions.length > 0 && (
38          <MentionDropdown
39            suggestions={suggestions}
40            onSelect={item => {
41              // 替换 @mention 为完整引用
42              const newText = text.replace(/@\w+\s+\S*$/, `@${item.path} `);
43              setText(newText);
44              setMentionQuery(null);
45            }}
46          />
47        )}
48      </div>
49    );
50  }
```

5.4 代码块渲染和一键插入

代码块

```
1   // CodeBlock.tsx
2   import { Prism as SyntaxHighlighter } from 'react-syntax-highlighter';
3   import { oneDark, oneLight } from 'react-syntax-
    highlighter/dist/esm/styles/prism';
4
5   interface CodeBlockProps {
6     code: string;
7     language: string;
8     theme: 'dark' | 'light';
9   }
10
11  function CodeBlock({ code, language, theme }: CodeBlockProps) {
12    const [copied, setCopied] = useState(false);
13
14    const handleCopy = () => {
15      vscode.postMessage({ type: 'copyCode', code });
16      setCopied(true);
17      setTimeout(() => setCopied(false), 2000);
18    };
19
20    const handleInsert = () => {
21      vscode.postMessage({ type: 'insertCode', code });
```

```
22    };
23
24    const handleNewFile = () => {
25      vscode.postMessage({ type: 'insertCode', code, createNewFile: true });
26    };
27
28    return (
29      <div className="code-block">
30        <div className="code-block-header">
31          <span className="language-label">{language}</span>
32          <div className="actions">
33            <button onClick={handleCopy} title="复制">
34              {copied ? '✓ 已复制' : '📋 复制'}
35            </button>
36            <button onClick={handleInsert} title="插入到光标位置">
37              📥 插入
38            </button>
39            <button onClick={handleNewFile} title="创建新文件">
40              📄 新文件
41            </button>
42          </div>
43        </div>
44        <SyntaxHighlighter
45          language={language}
46          style={theme === 'dark' ? oneDark : oneLight}
47          customStyle={{ margin: 0, borderRadius: '0 0 6px 6px' }}
48        >
49          {code}
50        </SyntaxHighlighter>
51      </div>
52    );
53  }
```

Extension Host 端处理插入:

代码块

```
1   async function insertCodeToEditor(code: string, filePath?: string) {
2     const editor = vscode.window.activeTextEditor;
3     if (!editor) {
4       vscode.window.showWarningMessage('没有打开的编辑器');
5       return;
6     }
```

```

  await editor.edit(editBuilder => {
    if (editor.selection.isEmpty) {
      // 在光标位置插入
      editBuilder.insert(editor.selection.active, code);
    } else {
      // 替换选中内容
      editBuilder.replace(editor.selection, code);
    }
  });
}
```

5.5 对话历史存储

代码块

```
// storage.ts
class ConversationStorage {
  private readonly storageKey = 'linkcode.conversations';

  constructor(
    private globalState: vscode.Memento, // context.globalState
    private maxConversations: number = 100,
  ) {}

  async saveConversation(conversation: Conversation): Promise<void> {
    const conversations = await this.getAllConversations();

    const index = conversations.findIndex(c => c.id === conversation.id);
    if (index >= 0) {
      conversations[index] = conversation;
    } else {
      conversations.unshift(conversation);
    }

    // 限制存储数量
    if (conversations.length > this.maxConversations) {
      conversations.splice(this.maxConversations);
    }

    await this.globalState.update(this.storageKey, conversations);
  }

```

```
  async getAllConversations(): Promise<Conversation[]> {
    return this.globalState.get<Conversation[]>(this.storageKey, []);
  }

  async deleteConversation(id: string): Promise<void> {
    const conversations = await this.getAllConversations();
    const filtered = conversations.filter(c => c.id !== id);
    await this.globalState.update(this.storageKey, filtered);
  }
}

interface Conversation {
  id: string;
  title: string;
  messages: ChatMessage[];
  createdAt: number;
  updatedAt: number;
  totalTokens: number;
  totalCost: number;
}

interface ChatMessage {
  id: string;
  role: 'user' | 'assistant' | 'system';
  content: string;
  model?: string;
  usage?: TokenUsage;
  attachments?: Attachment[];
  timestamp: number;
}
```

⚠ globalState 限制:

- `context.globalState` 存储在 SQLite 中，由 VS Code 管理
- 单个 key 的值大小建议 < 1MB
- 对于大量对话历史，建议定期清理或存储到文件
- P1 阶段可迁移到 Supabase 实现云端同步

5.6 Markdown 渲染

推荐库: `react-markdown` + `remark-gfm` + `rehype-highlight`

代码块

```
import ReactMarkdown from 'react-markdown';
import remarkGfm from 'remark-gfm';
import rehypeHighlight from 'rehype-highlight';

function MessageContent({ content }: { content: string }) {
  return (
    <ReactMarkdown
      remarkPlugins={[remarkGfm]}
      rehypePlugins={[rehypeHighlight]}
      components={{
        // 自定义代码块渲染
        code({ node, inline, className, children, ...props }) {
          const match = /language-(\w+)/.exec(className || '');

          if (!inline && match) {
            return (
              <CodeBlock
                code={String(children).replace(/\n$/, '')}
                language={match[1]}
                theme={currentTheme}
              />
            );
          }

          // 内联代码
          return (
            <code className="inline-code" {...props}>
              {children}
            </code>
          );
        },
        // 自定义链接
        a({ href, children }) {
          return (
            <a href="#" onClick={() => vscode.postMessage({ type: 'openUrl', url: href })}>
              {children}
            </a>
          );
        },
        // 自定义表格样式
        table({ children }) {
```

```
            return <table className="markdown-table">{children}</table>;
          },
        }}
      >
        {content}
      </ReactMarkdown>
    );
  }
```

Bundle 大小优化:

- `react-markdown`: ~50KB gzipped

- `remark-gfm`: ~5KB

- `react-syntax-highlighter`: ~300KB（仅导入需要的语言可减至 ~50KB）

- 推荐使用 `refractor` 的轻量版本按需加载语言

---

6. 项目索引和 RAG 方案

6.1 文件索引实现

代码块

```
class ProjectIndexer {
  private fileWatcher: vscode.FileSystemWatcher | null = null;
  private indexedFiles: Map<string, FileIndex> = new Map();
  private isIndexing = false;

  // 初始化：全量索引
  async initialize(workspaceFolder: vscode.WorkspaceFolder): Promise<void> {
    this.isIndexing = true;

    // 1. 查找所有代码文件
    const files = await vscode.workspace.findFiles(
      '**/*.{ts,tsx,js,jsx,py,java,go,rs,cpp,c,h,css,html,json,md,yaml,yml,toml}',
```

```
'{**/node_modules/**,**/.git/**,**/dist/**,**/build/**,**/.next/**,**/target/**}',
      10000  // 最多 10000 文件
    );

    // 2. 分批索引（避免阻塞）
    const batchSize = 50;
    for (let i = 0; i < files.length; i += batchSize) {
      const batch = files.slice(i, i + batchSize);
      await Promise.all(batch.map(uri => this.indexFile(uri)));

      // 每批后让出控制权
      await new Promise(resolve => setTimeout(resolve, 10));

      // 报告进度
      const progress = Math.round((i / files.length) * 100);
      vscode.window.setStatusBarMessage(`LinkCode: 索引中... ${progress}%`, 2000);
    }

    this.isIndexing = false;
    vscode.window.setStatusBarMessage(`LinkCode: 索引完成，共 ${files.length} 文件`, 5000);

    // 3. 启动文件变更监听（增量索引）
    this.startWatching();
  }

  private async indexFile(uri: vscode.Uri): Promise<void> {
    try {
      const content = await vscode.workspace.fs.readFile(uri);
      const text = new TextDecoder().decode(content);

      // 分块
      const chunks = this.chunkCode(text, uri.fsPath);

      // 生成 embedding
      const embeddings = await this.embeddingService.embed(chunks.map(c => c.content));

      this.indexedFiles.set(uri.toString(), {
        uri: uri.toString(),
        chunks: chunks.map((chunk, i) => ({
          ...chunk,
          embedding: embeddings[i],
        })),
```

```
          lastModified: Date.now(),
        });
      } catch (error) {
        // 忽略无法读取的文件
      }
    }

    // 增量索引
    private startWatching(): void {
      this.fileWatcher = vscode.workspace.create
```

代码块

```
FileSystemWatcher('**/*');

      this.fileWatcher.onDidChange(uri => {
        this.indexFile(uri); // 重新索引变更的文件
      });

      this.fileWatcher.onDidCreate(uri => {
        this.indexFile(uri);
      });

      this.fileWatcher.onDidDelete(uri => {
        this.indexedFiles.delete(uri.toString());
      });
    }

    dispose(): void {
      this.fileWatcher?.dispose();
    }
  }
```

6.2 Embedding 模型选择

| 方案 | 模型 | 维度 | 速度 | 成本 | 推荐 |

|------|------|------|------|------|------|

| API (Smoothlink) | text-embedding-3-small | 1536 | 中 | 低 | ✅ P0 默认 |

| API (Smoothlink) | text-embedding-3-large | 3072 | 慢 | 中 | P1 高精度 |

| 本地 ONNX | all-MiniLM-L6-v2 | 384 | 快 | 免费 | ✅ P1 离线 |

| 本地 ONNX | bge-small-zh-v1.5 | 512 | 快 | 免费 | 中文场景 |

推荐路径:

- MVP: 使用 Smoothlink API 的 embedding endpoint（简单、无额外依赖）

- P1: 引入本地 ONNX 模型（`@xenova/transformers` 或 `onnxruntime-node`），消除网络依赖

代码块

```
// 本地 Embedding 方案（P1）
import { pipeline } from '@xenova/transformers';

class LocalEmbeddingService {
  private model: any = null;

  async init() {
    // 首次运行会下载模型 (~23MB)
    this.model = await pipeline('feature-extraction', 'Xenova/all-MiniLM-L6-v2', {
      quantized: true, // 使用量化版本，更小更快
    });
  }

  async embed(texts: string[]): Promise<number[][]> {
    const results = await this.model(texts, {
      pooling: 'mean',
      normalize: true,
    });
    return Array.from(results.data).reduce((acc: number[][], _, i) => {
      if (i % 384 === 0) acc.push([]);
      acc[acc.length - 1].push(results.data[i]);
      return acc;
    }, []);
  }
}
```

6.3 向量存储方案

方案对比:

| 方案 | 依赖 | 性能 | 持久化 | 推荐 |

|------|------|------|--------|------|

| 内存 (暴力搜索) | 无 | 快(<10K向量) | ❌ | ✅ MVP |

| SQLite + sqlite-vss | sqlite3 | 中 | ✅ | ✅ P1 |

| LanceDB | lance WASM | 快 | ✅ | P1 备选 |

| Vectra (Node.js) | 无 | 中 | ✅ JSON | ✅ MVP-P1 过渡 |

MVP 推荐：内存向量搜索 + JSON 持久化

代码块

```
class SimpleVectorStore {
  private vectors: Map<string, { embedding: number[]; metadata: ChunkMetadata }> = new Map();
  private storePath: string;

  constructor(storagePath: string) {
    this.storePath = path.join(storagePath, 'vectors.json');
  }

  add(id: string, embedding: number[], metadata: ChunkMetadata): void {
    this.vectors.set(id, { embedding, metadata });
  }

  // 余弦相似度搜索
  search(queryEmbedding: number[], topK: number = 5): SearchResult[] {
    const results: SearchResult[] = [];

    for (const [id, { embedding, metadata }] of this.vectors) {
      const similarity = this.cosineSimilarity(queryEmbedding, embedding);
      results.push({ id, score: similarity, metadata });
    }

    results.sort((a, b) => b.score - a.score);
    return results.slice(0, topK);
  }

  private cosineSimilarity(a: number[], b: number[]): number {
    let dotProduct = 0;
    let normA = 0;
    let normB = 0;
    for (let i = 0; i < a.length; i++) {
```

```
31        dotProduct += a[i] * b[i];
32        normA += a[i] * a[i];
33        normB += b[i] * b[i];
34      }
35      return dotProduct / (Math.sqrt(normA) * Math.sqrt(normB));
36    }
37
38    // 持久化到磁盘
39    async save(): Promise<void> {
40      const data = Object.fromEntries(this.vectors);
41      await fs.promises.writeFile(this.storePath, JSON.stringify(data));
42    }
43
44    async load(): Promise<void> {
45      try {
46        const data = JSON.parse(await fs.promises.readFile(this.storePath, 'utf-
    8'));
47        this.vectors = new Map(Object.entries(data));
48      } catch {
49        // 文件不存在，从空开始
50      }
51    }
52  }
```

⚠ 性能注意:

- 10,000 个 384 维向量 ≈ 15MB 内存

- 暴力搜索 10K 向量约 1-5ms

- 超过 50K 向量应切换到 SQLite-vss 或 LanceDB

6.4 Chunk 策略

代码块

```
1  interface CodeChunk {
2    content: string;
3    filePath: string;
4    startLine: number;
5    endLine: number;
6    type: 'function' | 'class' | 'module' | 'block';
7    name?: string;
```

```
8   }
9
10  class CodeChunker {
11    // 按语义分块（使用 VS Code 的 DocumentSymbol）
12    async chunkBySymbols(document: vscode.TextDocument): Promise<CodeChunk[]> {
13      const chunks: CodeChunk[] = [];
14
15      const symbols = await
    vscode.commands.executeCommand<vscode.DocumentSymbol[]>(
16        'vscode.executeDocumentSymbolProvider',
17        document.uri
18      );
19
20      if (symbols && symbols.length > 0) {
21        this.extractSymbolChunks(document, symbols, chunks);
22      } else {
23        // 无符号信息时，回退到滑动窗口
24        chunks.push(...this.chunkByWindow(document));
25      }
26
27      return chunks;
28    }
29
30    private extractSymbolChunks(
31      document: vscode.TextDocument,
32      symbols: vscode.DocumentSymbol[],
33      chunks: CodeChunk[]
34    ): void {
35      for (const symbol of symbols) {
36        const text = document.getText(symbol.range);
37        const tokenEstimate = text.length / 3; // 粗略估计
38
39        if (tokenEstimate <= 500) {
40          // 小于 500 tokens，作为一个 chunk
41          chunks.push({
42            content: text,
43            filePath: document.uri.fsPath,
44            startLine: symbol.range.start.line,
45            endLine: symbol.range.end.line,
46            type: this.symbolKindToChunkType(symbol.kind),
47            name: symbol.name,
48          });
49        } else {
50          // 大于 500 tokens，递归处理子符号
51          if (symbol.children.length > 0) {
52            this.extractSymbolChunks(document, symbol.children, chunks);
53          } else {
```

```
            // 没有子符号，用滑动窗口分割
            const subChunks = this.splitLargeChunk(text, document.uri.fsPath, symbol.range.start.line);
            chunks.push(...subChunks);
          }
        }
      }
    }

    // 滑动窗口分块（fallback）
    private chunkByWindow(document: vscode.TextDocument): CodeChunk[] {
      const chunks: CodeChunk[] = [];
      const lines = document.getText().split('\n');
      const chunkSize = 50;  // 每 50 行一个 chunk
      const overlap = 5;     // 5 行重
```

代码块

```
叠

      for (let i = 0; i < lines.length; i += chunkSize - overlap) {
        const chunkLines = lines.slice(i, i + chunkSize);
        chunks.push({
          content: chunkLines.join('\n'),
          filePath: document.uri.fsPath,
          startLine: i,
          endLine: Math.min(i + chunkSize, lines.length),
          type: 'block',
        });
      }

      return chunks;
    }
  }
```

6.5 检索和重排序

代码块

```
class RAGRetriever {
  constructor(
    private vectorStore: SimpleVectorStore,
```

```
    private embeddingService: EmbeddingService,
  ) {}

  async retrieve(query: string, topK: number = 10): Promise<RetrievedChunk[]> {
    // Step 1: Embedding 查询
    const queryEmbedding = (await this.embeddingService.embed([query]))[0];

    // Step 2: 向量搜索（召回更多候选）
    const candidates = this.vectorStore.search(queryEmbedding, topK * 3);

    // Step 3: 重排序
    const reranked = this.rerank(query, candidates);

    // Step 4: 去重（同一文件的相邻 chunk 合并）
    const deduped = this.deduplicateAndMerge(reranked);

    return deduped.slice(0, topK);
  }

  // 简单重排序：结合向量相似度 + 关键词匹配
  private rerank(query: string, candidates: SearchResult[]): SearchResult[] {
    const queryTerms = query.toLowerCase().split(/\s+/);

    return candidates.map(candidate => {
      const content = candidate.metadata.content.toLowerCase();

      // 关键词匹配分数
      let keywordScore = 0;
      for (const term of queryTerms) {
        if (content.includes(term)) {
          keywordScore += 1 / queryTerms.length;
        }
      }

      // 文件名匹配加分
      const fileName = path.basename(candidate.metadata.filePath).toLowerCase();
      for (const term of queryTerms) {
        if (fileName.includes(term)) {
          keywordScore += 0.3;
        }
      }

      // 综合分数 = 0.7 * 向量相似度 + 0.3 * 关键词分数
      const combinedScore = 0.7 * candidate.score + 0.3 * keywordScore;

      return { ...candidate, score: combinedScore };
```

```
      }).sort((a, b) => b.score - a.score);
    }

    // 合并同文件相邻 chunk
    private deduplicateAndMerge(chunks: SearchResult[]): RetrievedChunk[] {
      const merged: RetrievedChunk[] = [];

      for (const chunk of chunks) {
        const last = merged[merged.length - 1];

        if (last &&
            last.filePath === chunk.metadata.filePath &&
            Math.abs(last.endLine - chunk.metadata.startLine) <= 5) {
          // 合并相邻 chunk
          last.content += '\n' +
```

代码块

```
  chunk.metadata.content;
          last.endLine = chunk.metadata.endLine;
          last.score = Math.max(last.score, chunk.score);
        } else {
          merged.push({
            content: chunk.metadata.content,
            filePath: chunk.metadata.filePath,
            startLine: chunk.metadata.startLine,
            endLine: chunk.metadata.endLine,
            score: chunk.score,
          });
        }
      }

      return merged;
    }
  }
```

---

7. Smoothlink API 集成细节

7.1 API 认证流程

方案 A：API Key 直连（MVP 推荐）:

代码块

```
class SmoothlinkClient {
  private apiKey: string;
  private baseUrl = 'https://api.smoothlink.ai';

  constructor(private authManager: AuthManager) {}

  async init(): Promise<void> {
    this.apiKey = await this.authManager.getApiKey() || '';

    // 监听 Key 变化
    this.authManager.onDidChange((e) => {
      if (e.key === 'linkcode.smoothlink.apiKey') {
        this.refreshApiKey();
      }
    });
  }

  private getHeaders(): Record<string, string> {
    return {
      'Authorization': `Bearer ${this.apiKey}`,
      'Content-Type': 'application/json',
      'X-Client': 'linkcode-vscode',
      'X-Client-Version': extensionVersion,
    };
  }
}
```

方案 B：OAuth2 登录（P1，更好的用户体验）:

代码块

```
class SmoothlinkOAuth {
  async login(): Promise<string> {
    // 1. 使用 VS Code Authentication API
```

```
    const session = await vscode.authentication.getSession(
      'smoothlink',  // provider ID
      ['read', 'write'],  // scopes
      { createIfNone: true }
    );

    return session.accessToken;
  }
}

// 注册 Authentication Provider
class SmoothlinkAuthProvider implements vscode.AuthenticationProvider {
  private sessions: vscode.AuthenticationSession[] = [];

  async getSessions(): Promise<readonly vscode.AuthenticationSession[]> {
    return this.sessions;
  }

  async createSession(scopes: readonly string[]): Promise<vscode.AuthenticationSession> {
    // 打开浏览器登录
    const callbackUri = await vscode.env.asExternalUri(
      vscode.Uri.parse('vscode://linkcode.linkcode/auth/callback')
    );

    // 打开 Smoothlink 登录页
    await vscode.env.openExternal(
      vscode.Uri.parse(`https://smoothlink.ai/oauth/authorize?` +
        `client_id=linkcode-vscode&` +
        `redirect_uri=${encodeURIComponent(callbackUri.toString())}&` +
        `scope=${scopes.join(' ')}&` +
        `response_type=code`)
    );

    // 等待回调
    const code = await this.waitForCallback();

    // 交换 token
    const tokenResponse = await this.exchangeCode(code);

    const session: vscode.AuthenticationSession = {
      id: crypto.randomUUID(),
      accessToken: tokenResponse.access_token,
      account: { id: tokenResponse.user_id, label: tokenResponse.email },
      scopes: [...scopes],
    };

```

```
    this.sessions.push(session);
    return session;
  }

  async removeSession(sessionId: string): Promise<void> {
    this.sessions = this.sessions.filter(s => s.id !== sessionId);
  }
}
```

7.2 Streaming 响应处理

代码块

```
class SmoothlinkClient {
  // 补全接口 (FIM)
  async *complete(params: CompletionParams): AsyncGenerator<string> {
    const response = await fetch(`${this.baseUrl}/v1/completions`, {
      method: 'POST',
      headers: this.getHeaders(),
      body: JSON.stringify({
        model: params.model,
        prompt: params.prompt,
        max_tokens: params.maxTokens || 128,
        temperature: params.temperature || 0.0,
        stop: params.stop || [],
        stream: true,
      }),
      signal: params.signal,
    });

    yield* this.parseSSEStream(response);
  }

  // Chat 接口
  async *chat(params: ChatParams): AsyncGenerator<ChatStreamChunk> {
    const response = await fetch(`${this.baseUrl}/v1/chat/completions`, {
      method: 'POST',
      headers: this.getHeaders(),
      body: JSON.stringify({
        model: params.model,
        messages: params.messages,
        max_tokens: params.maxTokens || 4096,
        temperature: params.temperature || 0.7,
```

```
31          stream: true,
32        }),
33        signal: params.signal,
34      });
35
36      if (!response.ok) {
37        const error = await response.text();
38        throw new SmoothlinkError(response.status, error);
39      }
40
41      yield* this.parseChatSSEStream(response);
42    }
43
44    // 通用 SSE 解析
45    private async *parseSSEStream(response: Response): AsyncGenerator<string> {
46      const reader = response.body!.getReader();
47      const decoder = new TextDecoder();
48      let buffer = '';
49
50      try {
51        while (true) {
52          const { done, value } = await reader.read();
53          if (done) break;
54
55          buffer += decoder.decode(value, { stream: true });
56          const lines = buffer.split('\n');
57          buffer = lines.pop() || '';
58
59          for (const line of lines) {
60            if (!line.startsWith('data: ')) continue;
61            const data = line.slice(6).trim();
62            if (data === '[DONE]') return;
63
64            try {
65              const parsed = JSON.parse(data);
66              const text = parsed.choices?.[0]?.text
```

代码块

```
1   || parsed.choices?.[0]?.delta?.content;
2               if (text) yield text;
3             } catch {}
4           }
5         }
6       } finally {
7         reader.releaseLock();
```

```
      }
    }
  }
```

7.3 错误处理和重试策略

代码块

```
class RetryableClient {
  private readonly maxRetries = 3;
  private readonly baseDelay = 1000; // 1 秒

  async fetchWithRetry(url: string, options: RequestInit): Promise<Response> {
    let lastError: Error | null = null;

    for (let attempt = 0; attempt <= this.maxRetries; attempt++) {
      try {
        const response = await fetch(url, {
          ...options,
          signal: this.createTimeoutSignal(options.signal as AbortSignal, 30000),
        });

        // 成功或不可重试的错误
        if (response.ok || !this.isRetryable(response.status)) {
          return response;
        }

        // 可重试的错误
        lastError = new SmoothlinkError(response.status, await response.text());

        // 429: Rate Limited - 使用 Retry-After header
        if (response.status === 429) {
          const retryAfter = parseInt(response.headers.get('Retry-After') || '5');
          await this.delay(retryAfter * 1000);
          continue;
        }

      } catch (error: any) {
        if (error.name === 'AbortError') throw error;
        lastError = error;
```

```
      }

      // 指数退避
      if (attempt < this.maxRetries) {
        const delay = this.baseDelay * Math.pow(2, attempt) + Math.random() * 1000;
        await this.delay(delay);
      }
    }

    throw lastError || new Error('Max retries exceeded');
  }

  private isRetryable(status: number): boolean {
    return [429, 500, 502, 503, 504].includes(status);
  }

  private createTimeoutSignal(existingSignal: AbortSignal | undefined, timeoutMs: number): AbortSignal {
    const controller = new AbortController();
    const timeout = setTimeout(() => controller.abort(), timeoutMs);

    if (existingSignal) {
      existingSignal.addEventListener('abort', () => {
        clearTimeout(timeout);
        controller.abort();
      });
    }

    return controller.signal;
  }

  private delay(ms: number): Promise<void> {
    return new Promise(resolve => setTimeout(resolve, ms));

```

代码块

```
}
}
```

7.4 Token 计费实时追踪

代码块

```
class UsageTracker {
  private sessionUsage: UsageRecord = { totalTokens: 0, totalCost: 0, requests: 0 };
  private dailyUsage: UsageRecord = { totalTokens: 0, totalCost: 0, requests: 0 };

  // 状态栏显示
  private statusBarItem: vscode.StatusBarItem;

  constructor() {
    this.statusBarItem = vscode.window.createStatusBarItem(vscode.StatusBarAlignment.Right, 100);
    this.statusBarItem.command = 'linkcode.showUsageDetail';
    this.updateStatusBar();
    this.statusBarItem.show();
  }

  recordUsage(usage: TokenUsage, model: string): void {
    const cost = this.calculateCost(usage, model);

    this.sessionUsage.totalTokens += usage.totalTokens;
    this.sessionUsage.totalCost += cost;
    this.sessionUsage.requests++;

    this.dailyUsage.totalTokens += usage.totalTokens;
    this.dailyUsage.totalCost += cost;
    this.dailyUsage.requests++;

    this.updateStatusBar();

    // 通知 Webview 更新余额
    this.notifyBalanceChange();
  }

  private calculateCost(usage: TokenUsage, model: string): number {
    const pricing = MODEL_PRICING[model];
    if (!pricing) return 0;

    return (usage.promptTokens / 1_000_000) * pricing.inputPrice +
           (usage.completionTokens / 1_000_000) * pricing.outputPrice;
  }

  private updateStatusBar(): void {
```

```
    const costStr = this.sessionUsage.totalCost < 0.01
      ? `< ¥0.01`
      : `¥${(this.sessionUsage.totalCost * 7.2).toFixed(2)}`; // USD to CNY

    this.statusBarItem.text = `$(zap) LinkCode: ${costStr}`;
    this.statusBarItem.tooltip =
      `本次会话: ${this.sessionUsage.requests} 次请求\n` +
      `消耗 Tokens: ${this.sessionUsage.totalTokens.toLocaleString()}\n` +
      `费用: ${costStr}`;
  }

  // 从 Smoothlink API 查询余额
  async fetchBalance(): Promise<{ balance: number; currency: string }> {
    const response = await this.client.get('/v1/user/balance');
    return response.data;
  }
}

// 模型定价表 ($/M tokens)
const MODEL_PRICING: Record<string, { inputPrice: number; outputPrice: number
}> =
```

代码块

```
 {
  'deepseek-coder-v2-lite': { inputPrice: 0.1, outputPrice: 0.1 },
  'deepseek-coder-v2': { inputPrice: 0.14, outputPrice: 0.28 },
  'deepseek-v3': { inputPrice: 0.27, outputPrice: 1.10 },
  'claude-sonnet-4-20250514': { inputPrice: 3.0, outputPrice: 15.0 },
  'qwen2.5-coder-32b': { inputPrice: 0.3, outputPrice: 0.6 },
};
```

7.5 Rate Limiting 处理

代码块

```
class RateLimiter {
  private requestTimestamps: number[] = [];
  private readonly maxRequestsPerMinute = 60;  // Free plan
  private readonly maxRequestsPerSecond = 10;

  async waitForSlot(): Promise<void> {
```

```
    const now = Date.now();

    // 清理过期记录
    this.requestTimestamps = this.requestTimestamps.filter(t => now - t < 60000);

    // 检查每分钟限制
    if (this.requestTimestamps.length >= this.maxRequestsPerMinute) {
      const oldestInMinute = this.requestTimestamps[0];
      const waitMs = 60000 - (now - oldestInMinute);
      if (waitMs > 0) {
        await new Promise(resolve => setTimeout(resolve, waitMs));
      }
    }

    // 检查每秒限制
    const recentRequests = this.requestTimestamps.filter(t => now - t < 1000);
    if (recentRequests.length >= this.maxRequestsPerSecond) {
      await new Promise(resolve => setTimeout(resolve, 100));
    }

    this.requestTimestamps.push(Date.now());
  }
}
```

---

8. 安全和隐私方案

8.1 API Key 存储完整方案

代码块

```
class SecureKeyManager {
  constructor(private secrets: vscode.SecretStorage) {}

  // 存储流程
  async setApiKey(key: string): Promise<void> {
    // 1. 验证 key 格式
    if (!this.validateKeyFormat(key)) {
```

```
8         throw new Error('Invalid API key format');
9       }
10
11      // 2. 验证 key 有效性（调用一次 API）
12      const isValid = await this.verifyKey(key);
13      if (!isValid) {
14        throw new Error('API key is invalid or expired');
15      }
16
17      // 3. 存储到 Secret Storage
18      await this.secrets.store('linkcode.apiKey', key);
19
20      // 4. 清除内存中的明文
21      // (JavaScript 无法强制清除字符串，但可以取消引用)
22    }
23
24    // 获取流程
25    async getApiKey(): Promise<string | undefined> {
26      return this.secrets.get('linkcode.apiKey');
27    }
28
29    private validateKeyFormat(key: string): boolean {
30      // Smoothlink API key 格式检查
31      return /^sk-[a-zA-Z0-9]{32,}$/.test(key);
32    }
33
34    private async verifyKey(key: string): Promise<boolean> {
35      try {
36        const response = await fetch('https://api.smoothlink.ai/v1/models', {
37          headers: { 'Authorization': `Bearer ${key}` },
38        });
39        return response.ok;
40      } catch {
41        return false;
42      }
43    }
44  }
```

安全检查清单:

- [x] API Key 存储在 VS Code Secret Storage（系统级密钥管理）
- [x] API Key 不写入 settings.json、不写入日志
- [x] API Key 不通过 Webview postMessage 传递（只在 Extension Host 中使用）

- [x] HTTPS 通信（所有 API 调用）
- [x] 不在错误信息中暴露 API Key

8.2 代码不外泄的技术保障

代码块

```
// 代码隐私策略
class PrivacyGuard {
  // 默认配置
  private config = {
    sendCodeToAPI: true,           // 补全/Chat 需要发送代码
    cacheCodeLocally: false,       // 不本地持久化代码内容
    telemetryIncludesCode: false,  // 遥测不包含代码
    respectGitignore: true,        // 尊重 .gitignore
  };

  // 检查文件是否应该被排除
  shouldExcludeFile(filePath: string): boolean {
    // 1. 检查 .gitignore
    if (this.config.respectGitignore && this.isGitIgnored(filePath)) {
      return true;
    }

    // 2. 检查 .linkcodeIgnore 自定义排除
    if (this.isLinkcodeIgnored(filePath)) {
      return true;
    }

    // 3. 排除敏感文件
    const sensitivePatterns = [
      /\.env$/,
      /\.env\.\w+$/,
      /secrets?\.\w+$/,
      /credentials?\.\w+$/,
      /\.pem$/,
      /\.key$/,
      /id_rsa/,
    ];

    return sensitivePatterns.some(p => p.test(filePath));
  }

  // 清理发送给 API 的代码（移除敏感信息）
```

```
  sanitizeForAPI(code: string): string {
    // 移除硬编码的密钥/密码
    return code
      .replace(/(password|secret|token|key|api_key)\s*[:=]\s*['"][^'"]+['"]/gi,
                '$1 = "[REDACTED]"')
      .replace(/Bearer\s+[a-zA-Z0-9\-._~+/]+=*/g, 'Bearer [REDACTED]');
  }
}
```

`.linkcodeIgnore` 文件格式 (类似 .gitignore):

代码块

```
# 不发送给 AI 的文件/目录
.env*
secrets/
config/production.json
*.pem
*.key
```

8.3 Telemetry 收集边界

代码块

```
// 收集的数据
interface TelemetryEvent {
  // ✅ 收集
  eventType: string;           // 事件类型 (completion_accepted, chat_sent, etc.)
  model: string;               // 使用的模型
  latencyMs: number;           // 响应延迟
  tokenCount: number;          // token 数量
  language: string;            // 编程语言
  completionLength: number;    // 补全长度（字符数）
  accepted: boolean;           // 是否接受
  errorType?: string;          // 错误类型

  // ❌ 不收集
  // codeContent: string;      // 用户代码内容
```

```
    // fileName: string;          // 文件名
    // chatMessage: string;       // 聊天消息内容
    // apiKey: string;            // API 密钥
  }

  class TelemetryService {
    private enabled: boolean;

    constructor() {
      // 尊重 VS Code 全局遥测设置
      this.enabled = vscode.env.isTelemetryEnabled;

      // 也检查 LinkCode 自己的设置
      const config = vscode.workspace.getConfiguration('linkcode');
      this.enabled = this.enabled && config.get('telemetry.enabled', true);
    }

    track(event: TelemetryEvent): void {
      if (!this.enabled) return;

      // 不包含任何代码内容或个人信息
      // 仅发送使用统计
    }
  }
```

---

9. 性能优化方案

9.1 补全延迟优化 (P90 < 500ms)

延迟分解:

代码块

```
总延迟 = Debounce(150-300ms) + 上下文构建(5-20ms) + 网络RTT(50-200ms) + 模型推理(100-500ms) + 后处理(1-5ms)
```

目标: 用户感知延迟（从停止输入到看到 Ghost Text）< 500ms

优化策略:

策略	预期收益	实现难度
智能 Debounce（行尾更短）	-100ms	低
缓存命中	-300ms (95%命中时)	低
预请求（打字时预测）	-200ms	中
使用更快的模型（Lite）	-200ms	低
连接复用 (HTTP/2 Keep-Alive)	-50ms	低
选择最近的 API 节点	-50ms	低

代码块

```
// 预请求策略
class PrefetchManager {
  private prefetchCache: Map<string, Promise<string>> = new Map();

  // 在用户输入"当前行"时，预先请求"下一行"的补全
  onDocumentChange(document: vscode.TextDocument, position: vscode.Position): void {
    // 检查是否在行尾
    const line = document.lineAt(position.line).text;
    if (position.character < line.trimEnd().length) return;

    // 预取下一行补全
    const prefetchKey = `${document.uri}:${position.line + 1}`;
    if (!this.prefetchCache.has(prefetchKey)) {
      const promise = this.fetchCompletion(document, new vscode.Position(position.line + 1, 0));
      this.prefetchCache.set(prefetchKey, promise);

      // 10 秒后过期
      setTimeout(() => this.prefetchCache.delete(prefetchKey), 10000);
    }
  }
}
```

连接优化:

代码块

```
import { Agent } from 'https';

// HTTP Keep-Alive
const httpsAgent = new Agent({
  keepAlive: true,
  keepAliveMsecs: 30000,
  maxSockets: 5,
  maxFreeSockets: 2,
});

// 或使用 undici (Node.js 内置高性能 HTTP 客户端)
import { Pool } from 'undici';
const pool = new Pool('https://api.smoothlink.ai', {
  connections: 5,
  pipelining: 1,
  keepAliveTimeout: 30000,
});
```

9.2 Extension 内存控制

代码块

```
class MemoryManager {
  private readonly MAX_MEMORY_MB = 200;

  // 定期检查内存使用
  startMonitoring(): void {
    setInterval(() => {
      const usage = process.memoryUsage();
      const heapMB = usage.heapUsed / 1024 / 1024;

      if (heapMB > this.MAX_MEMORY_MB * 0.8) {
        console.warn(`[LinkCode] High memory usage: ${heapMB.toFixed(1)}MB`);
        this.cleanup();
      }
    }, 30000); // 每 30 秒检查
  }
```

```

  private cleanup(): void {
    // 1. 清理补全缓存
    this.completionCache.clear();

    // 2. 清理过期的向量索引
    this.vectorStore.evictOldEntries(3600000); // 1 小时前的

    // 3. 清理对话历史中的旧消息（只保留最近 50 条在内存）
    this.chatHistory.truncate(50);

    // 4. 触发 GC
    if (global.gc) {
      global.gc();
    }
  }
}

// 大数据操作使用 Worker Thread
import { Worker } from 'worker_threads';

class IndexWorker {
  private worker: Worker;

  constructor() {
    this.worker = new Worker(path.join(__dirname, 'indexWorker.js'));
  }

  async indexFiles(files: string[]): Promise<void> {
    return new Promise((resolve, reject) => {
      this.worker.postMessage({ type: 'index', files });
      this.worker.once('message', (msg) => {
        if (msg.type === 'done') resolve();
        else reject(new Error(msg.error));
      });
    });
  }
}
```

9.3 大项目 (10000+ 文件) 索引性能

策略:

1. **优先索引**:
   - 最近修改的文件优先
   - 打开的文件优先
   - 源代码文件优先（排除生成文件、assets 等）

2. **延迟索引**:
   - 初始只索引文件元信息（路径、大小、修改时间）
   - 用户首次 @workspace 查询时才触发全量索引
   - 后台空闲时渐进索引

3. **索引配置**:

代码块

```
{
  "linkcode.indexing.maxFiles": 10000,
  "linkcode.indexing.excludePatterns": [
    "**/node_modules/**",
    "**/dist/**",
    "**/.git/**",
    "**/build/**",
    "**/*.min.js",
    "**/vendor/**"
  ],
  "linkcode.indexing.includePatterns": [
    "**/*.ts", "**/*.tsx", "**/*.js", "**/*.jsx",
    "**/*.py", "**/*.java", "**/*.go", "**/*.rs",
    "**/*.cpp", "**/*.c", "**/*.h"
  ],
  "linkcode.indexing.maxFileSize": 524288
}
```

4. **性能目标**:

项目规模	索引时间	内存占用	查询延迟

| 1K 文件 | < 10s | ~30MB | < 50ms |

| 5K 文件 | < 30s | ~80MB | < 100ms |

| 10K 文件 | < 60s | ~150MB | < 200ms |

| 50K 文件 | < 5min | ~300MB | < 500ms |

---

10. 发布和分发方案

10.1 VS Code Marketplace 发布流程

准备工作:

1. **Azure DevOps 账户**: 需要创建个人访问令牌 (PAT)
2. **Publisher**: 在 https://marketplace.visualstudio.com/manage 注册 publisher
3. **vsce**: 官方打包工具

代码块

```
# 安装打包工具
npm install -g @vscode/vsce

# 创建 publisher
vsce create-publisher linkcode

# 打包
vsce package

# 发布
vsce publish -p <personal-access-token>

# 或使用 CI/CD (推荐)
```

package.json 关键字段:

代码块

```
{
  "name": "linkcode",
  "displayName": "LinkCode - AI 多模型编程助手",
  "description": "基于 61+ AI 模型的智能编程助手，智能路由选择最优模型",
  "version": "0.1.0",
  "publisher": "linkcode",
  "engines": {
    "vscode": "^1.85.0"
  },
  "categories": ["AI", "Programming Languages", "Snippets"],
  "keywords": ["ai", "copilot", "autocomplete", "deepseek", "claude", "中文"],
  "activationEvents": ["onStartupFinished"],
  "main": "./dist/extension.js",
  "icon": "images/icon.png",
  "galleryBanner": {
    "color": "#0066FF",
    "theme": "dark"
  },
  "repository": {
    "type": "git",
    "url": "https://github.com/linkcode/linkcode-vscode"
  },
  "contributes": {
    "configuration": {
      "title": "LinkCode",
      "properties": {
        "linkcode.routing.preferCheap": {
          "type": "boolean",
          "default": false,
          "description": "优先选择更便宜的模型"
        }
      }
    },
    "viewsContainers": {
      "activitybar": [
        {
          "id": "linkcode",
          "title": "LinkCode",
          "icon": "images/sidebar-icon.svg"
        }
      ]
    },
    "views": {
```

```
        "linkcode": [
          {
            "type": "webview",
            "id": "linkcode.chatView",
            "name": "Chat"
          }
        ]
      },
      "commands": [
        {
          "command": "linkcode.openChat",
          "title": "LinkCode: 打开 AI 对话"
        },
        {
          "command": "linkcode.setApiKey",
          "title": "LinkCode: 设置 API Key"
        }
      ],
      "keybindings": [
        {
          "command": "linkcode.openChat",
          "key": "ctrl+shift+l",
          "mac": "cmd+shift+l"
        }
      ]
    }
  }
```

审核要点:

- ⚠ 扩展名和描述不能误导（不能叫 "Copilot" 或暗示是微软产品）
- ⚠ 必须有隐私政策链接
- ⚠ 不能 bundle 恶意代码
- ⚠ 不能过度请求权限
- ✅ 需要提供详细的 README.md（含截图）
- ✅ 需要 CHANGELOG.md
- ✅ 图标必须是 128x128 PNG
- ✅ 包大小建议 < 20MB

10.2 自动更新机制

VS Code Marketplace 原生支持自动更新：

- 用户安装后，VS Code 会定期检查更新

- 新版本发布到 Marketplace 后，用户会在 1-24h 内收到更新提示

- 可以在 `package.json` 中设置 `engines.vscode` 限制最低 VS Code 版本

增量更新: VS Code Marketplace 不支持增量更新，每次都是完整下载。

10.3 版本管理策略

代码块

```
语义化版本号: MAJOR.MINOR.PATCH
- MAJOR: 破坏性变更 (API Key 格式变化等)
- MINOR: 新功能
- PATCH: Bug 修复

0.1.x: MVP 内测
0.2.x-0.9.x: 功能迭代
1.0.0: 正式发布

Pre-release 版本:
0.1.0-beta.1  (VS Code Marketplace 支持 pre-release channel)
```

CI/CD 发布流程:

代码块

```yaml
# .github/workflows/release.yml
name: Release
on:
  push:
    tags:
      - 'v*'
```

```yaml

jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm ci
      - run: npm run build
      - run: npm run test
      - run: npx vsce package
      - run: npx vsce publish -p ${{ secrets.VSCE_PAT }}

      # 同时发布到 Open VSX (for VS Codium 等开源 VS Code)
      - run: npx ovsx publish -p ${{ secrets.OVSX_PAT }}
```

10.4 企业自分发方案 (VSIX)

代码块

```
# 打包为 VSIX 文件
vsce package --out linkcode-enterprise-1.0.0.vsix

# 企业用户安装
code --install-extension linkcode-enterprise-1.0.0.vsix

# 或通过 VS Code GUI: Extensions → ... → Install from VSIX
```

企业分发渠道:

1. **内部下载页面**: 提供 VSIX 文件下载
2. **私有 Extension Gallery**: 使用 `open-vsx` 搭建私有 marketplace
3. **Settings Sync**: 通过企业 MDM 推送 extensions.json
4. **Docker Dev Container**: 在 `.devcontainer/devcontainer.json` 中预装

```
// .devcontainer/devcontainer.json
{
  "customizations": {
    "vscode": {
      "extensions": [
        "linkcode.linkcode"
      ],
      "settings": {
        "linkcode.apiKey": "${localEnv:LINKCODE_API_KEY}"
      }
    }
  }
}
```

附录 A: 推荐技术栈总结

层次	技术选型	版本	备注
Extension 框架	VS Code Extension API	^1.85.0	
语言	TypeScript	~5.4	strict mode
打包	esbuild	^0.20	比 webpack 快 10x
Webview UI	React	^18.2	
UI 样式	TailwindCSS	^3.4	
Markdown	react-markdown	^9.0	
代码高亮	react-syntax-highlighter	^15.5	或 shiki
HTTP Client	Node.js fetch (内置)	-	Node 18+
向量存储(MVP)	自实现内存存储	-	
向量存储(P1)	LanceDB	^0.4	或 sqlite-vss
Embedding(MVP)	Smoothlink API	-	
Embedding(P1)	@xenova/transformers	^2.17	本地 ONNX
AST 解析(P1)	web-tree-sitter	^0.22	

| 测试 | vitest | ^1.5 | |

| E2E 测试 | @vscode/test-electron | ^2.3 | |

附录 B: 风险矩阵

| 风险 | 概率 | 影响 | 缓解方案 |

|------|------|------|----------|

| Smoothlink API 不稳定 | 中 | 高 | 多模型 fallback + 本地缓存 |

| 补全延迟 > 500ms | 高 | 高 | 用轻量模型、预请求、缓存 |

| 与 Copilot 冲突 | 高 | 中 | 文档说明、检测冲突并提示 |

| Webview 内存泄漏 | 中 | 中 | 定期监控、状态管理规范 |

| VS Code API 变更 | 低 | 高 | 使用 stable API、版本锁定 |

| 大项目索引 OOM | 中 | 高 | Worker Thread、增量索引、内存监控 |

| API Key 泄露 | 低 | 极高 | Secret Storage、代码审计 |

| Marketplace 审核不通过 | 低 | 中 | 提前了解规则、准备完整文档 |

文档结束 - LinkCode VS Code 插件深度实现细节调研报告 v1.0